OSCAR -
Open Source Copter for Academic Research

sddec26-05

By: Paul Zell, Jessica Reiff, Grace White, Chloe Atwood, Gracie Finger, Collin Van Weelden, Matthew Smith

OSCAR - Open Source Copter for Academic Research

Client: Self-proposed

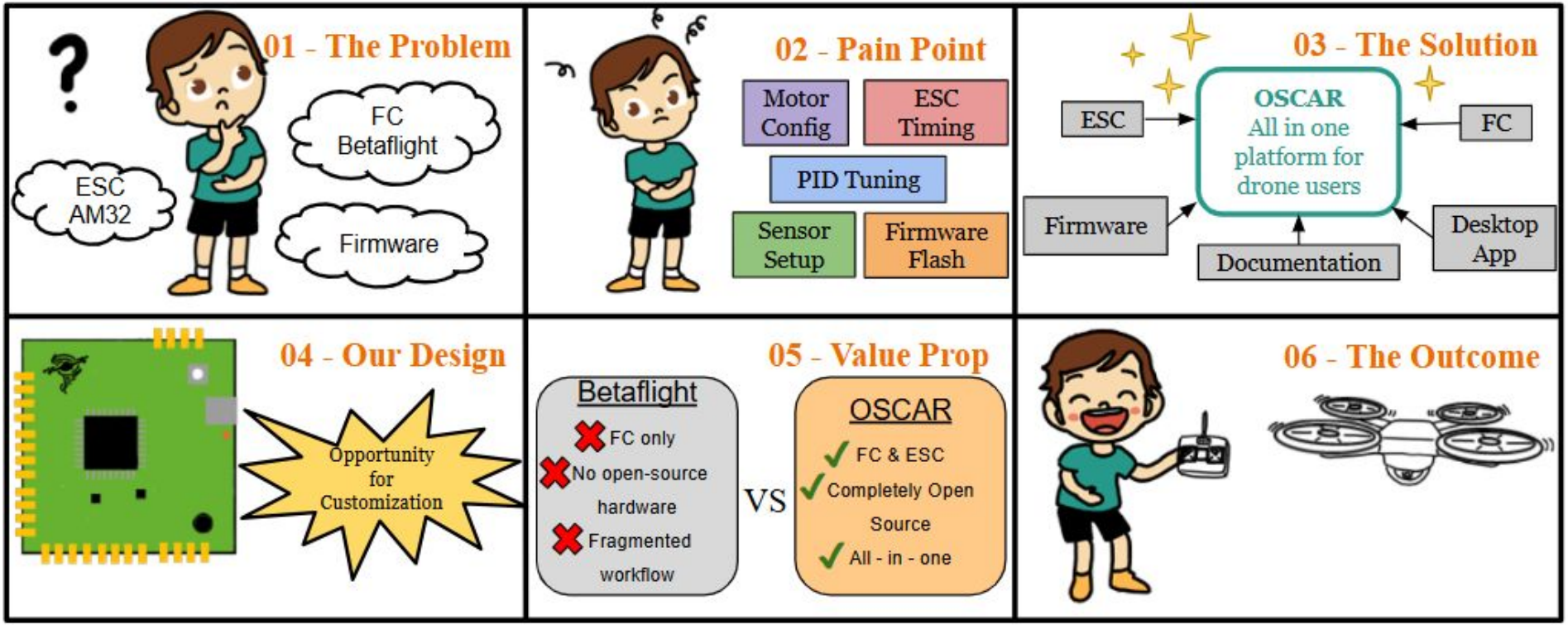
Advisor: Dr. Nathan Neihart

- Completely open source custom drone software down to hardware
- Targeted towards researchers, students, and hobbyists



OSCAR Emblem

StoryBoard


01 - The Problem
?
FC
Betaflight
ESC
AM32
Firmware
02 - Pain Point
Motor
Config
ESC
Timing
PID Tuning
Sensor
Setup
Firmware
Flash
03 - The Solution
ESC
OSCAR
All in one
platform for
drone users
FC
Firmware
Documentation
Desktop
App
04 - Our Design
Opportunity
for
Customization
05 - Value Prop
Betaflight
FC only
No open-source
hardware
Fragmented
workflow
VS
OSCAR
FC & ESC
Completely Open
Source
All - in - one
06 - The Outcome

High-Level Design Requirements

(1): Open Source

- Replaces black box flight stacks
- Fully documented hardware, firmware, and software

(2): Flight Capability

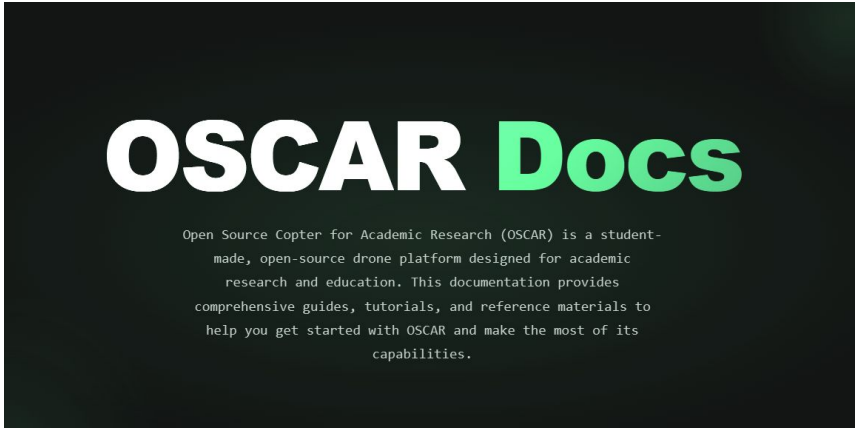
- 5 inch drone frame powered by a 4S LiPo battery (14.8V nominal)
- Stable level flight using IMU and magnetometer (10 seconds)
    - 8KHz control loop
    - 99% valid communication packets across protocols

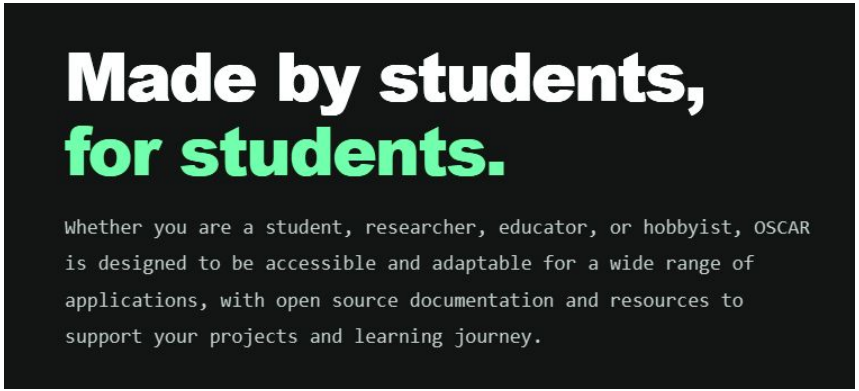
Open-Source Contributions

Documentation Website

Documentation website hosted by GitHub Pages with the following goals:

- Ease of navigation
- Visually pleasing
- Detailed instructions
- Access to datasheets, external resources, and our public project GitHub repo





https://gracefinger.github.io/OSCAR_Documentation.github.io/index.html

Application Executable & Distribution

Build & Packaging

- Desktop executable (.exe)
- Built using Electron Forge and Squirrel

Distribution

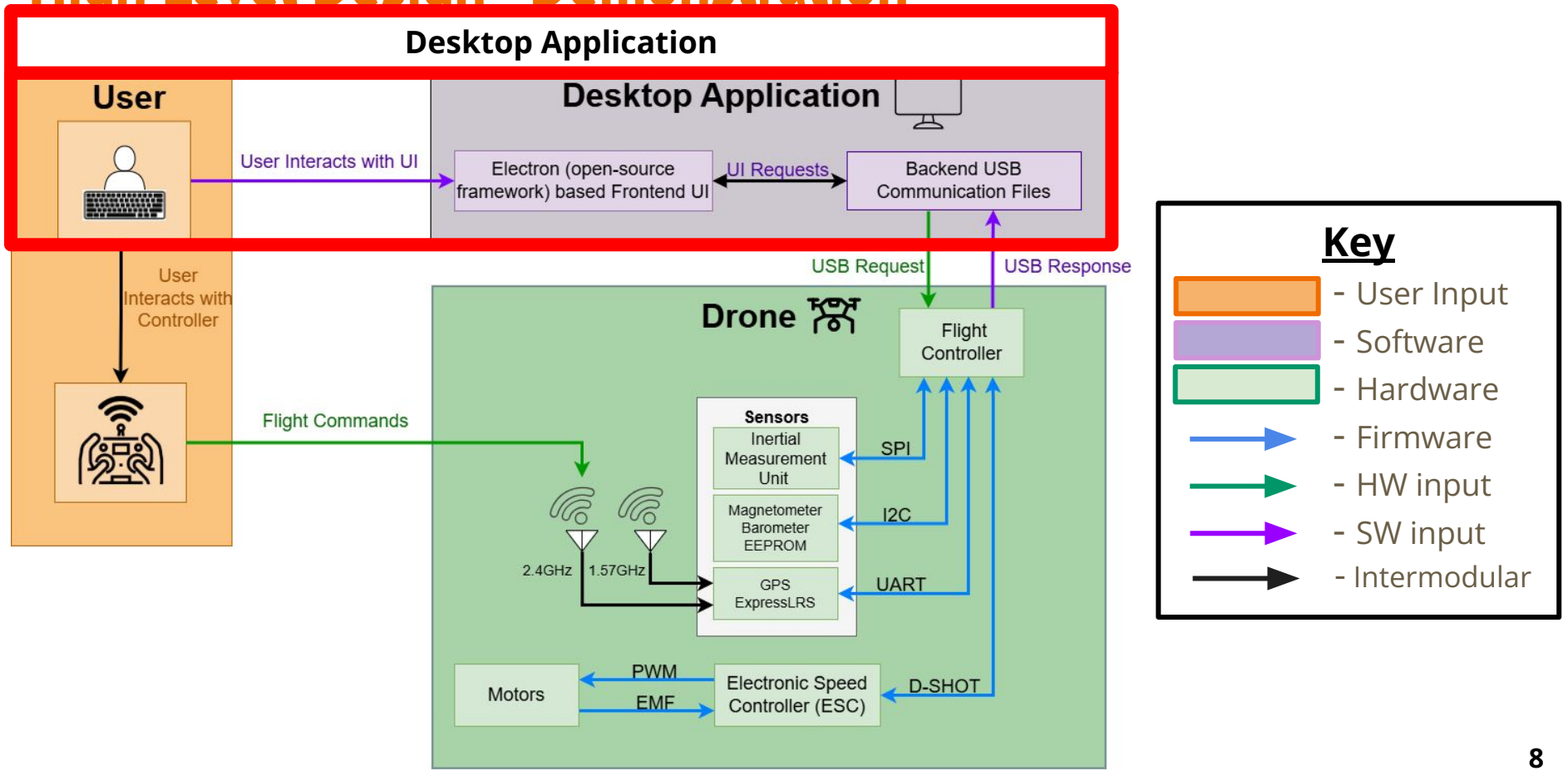
- Distributed as a standalone installer
- All dependencies bundled in Application

System Requirements

- Windows machine
- USB connection to flight controller

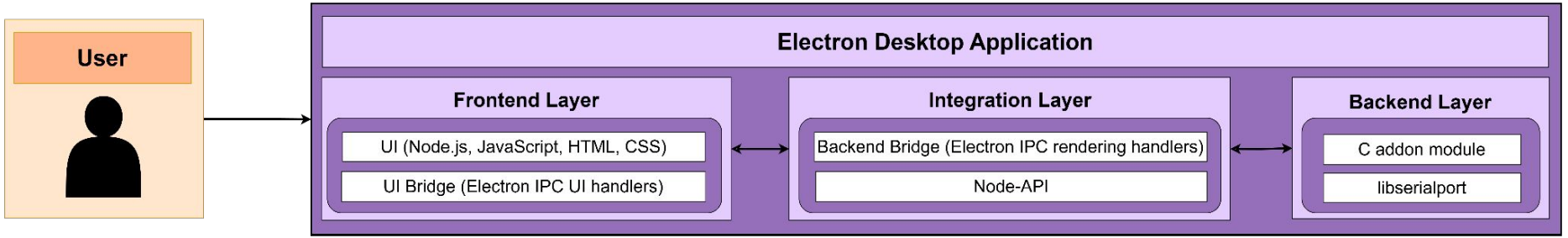


High Level Design - Demonstration



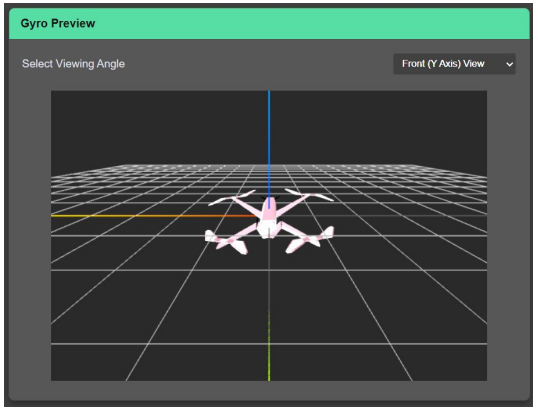
Desktop Application - Design Overview

- System Architecture:
  - Frontend Layer: Electron (Open-Source Framework for Desktop UI)
  - Integration Layer: Node-API connects the C backend to the Electron frontend
  - Backend Layer: C addon using libserialport



Desktop Application - User Interface Design

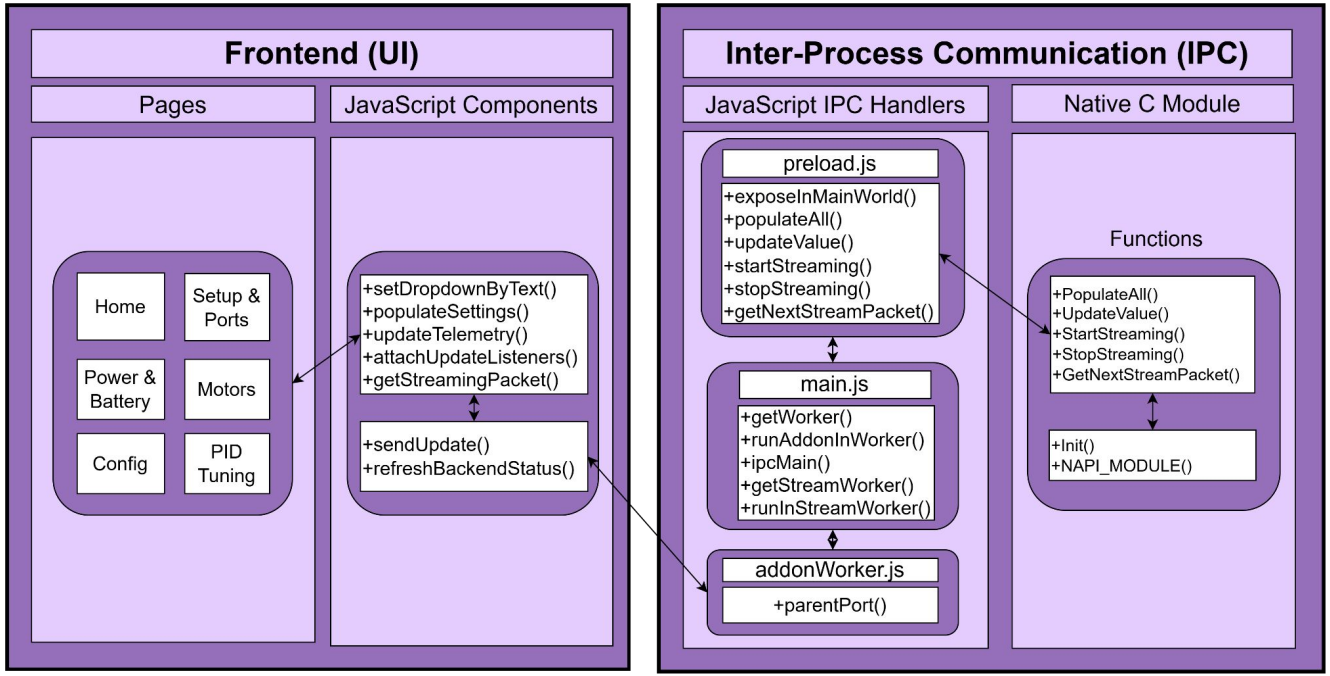
- Custom drone settings inspired by Betaflight (open-source flight control system)
- UI built through standard web technologies, and executed within the Electron Renderer process
  - CSS - Styling and Layout
  - HTML - Content Structure
  - JavaScript - Dynamic Behavior, Event Handling, Backend Communication





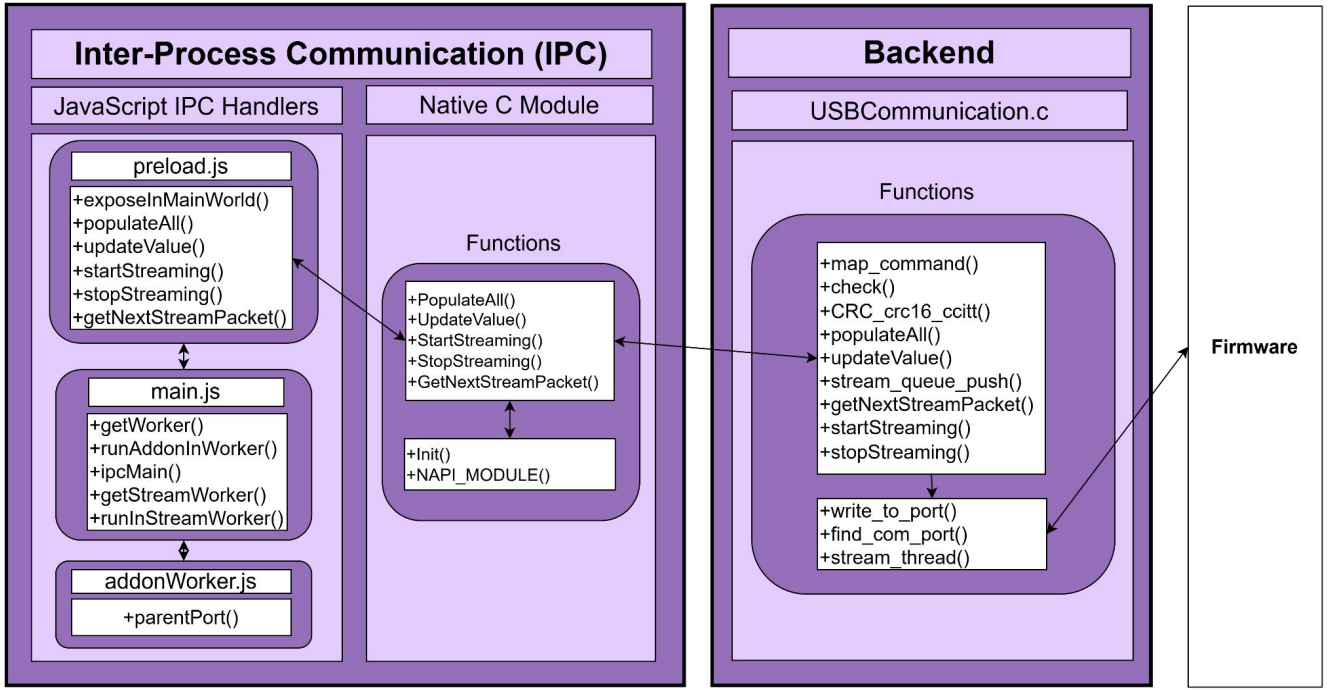
Desktop Application - Frontend

- Frontend Responsibilities
  - Provides interactive user interface
  - Handles user interactions
  - Displays drone state and feedback
  - Sends commands to backend through inter-process communication mechanisms
- Uses multithreading to control separation of UI tasks vs command and response related tasks.

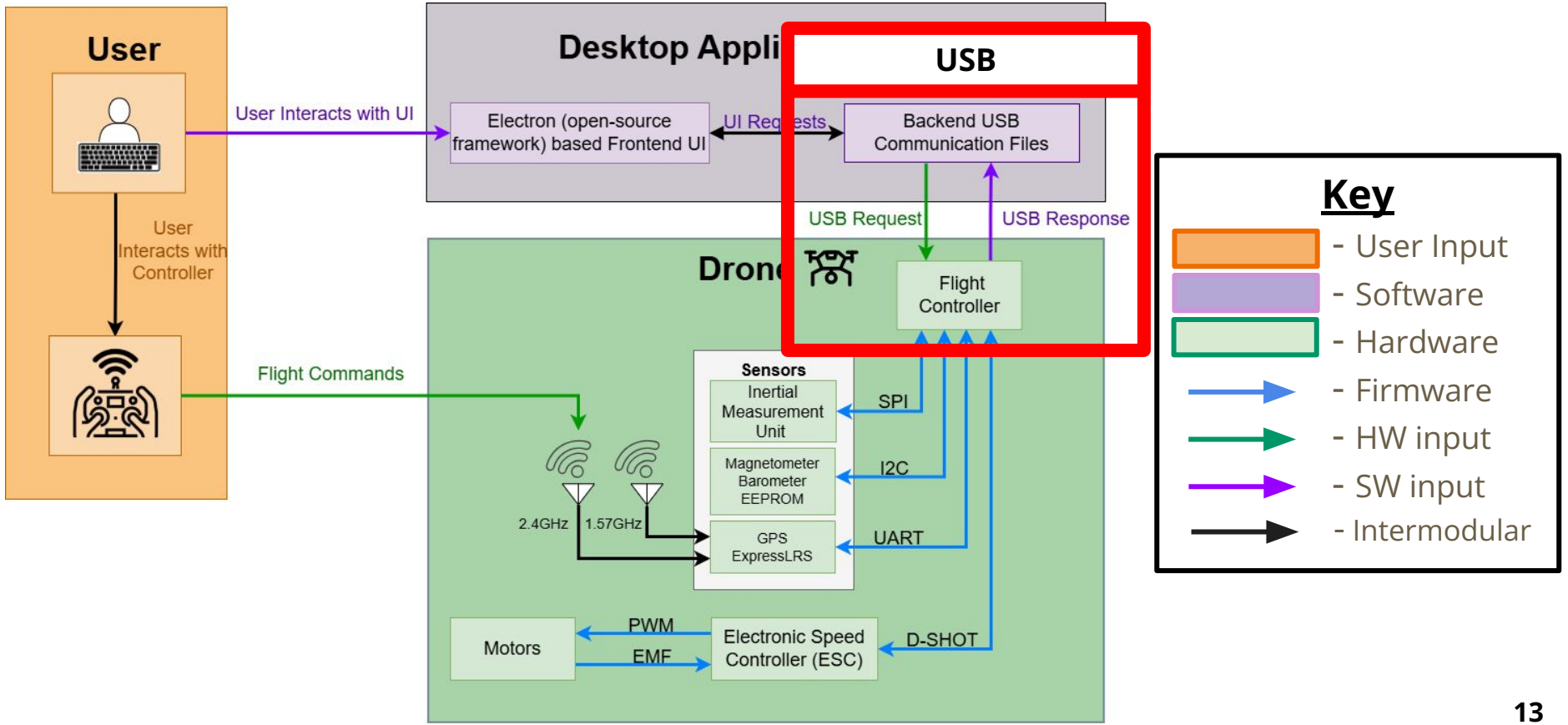


Desktop Application - Backend

- Backend Responsibilities
  - Processes application commands
  - Handles data between UI and hardware
  - Validates data received from hardware
  - Returns processed data back to frontend
- Multithreading and locks manage data communication and flow

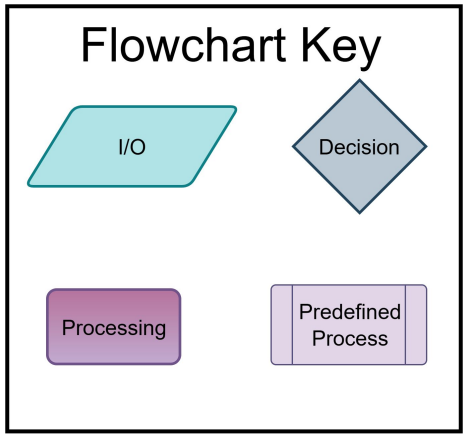


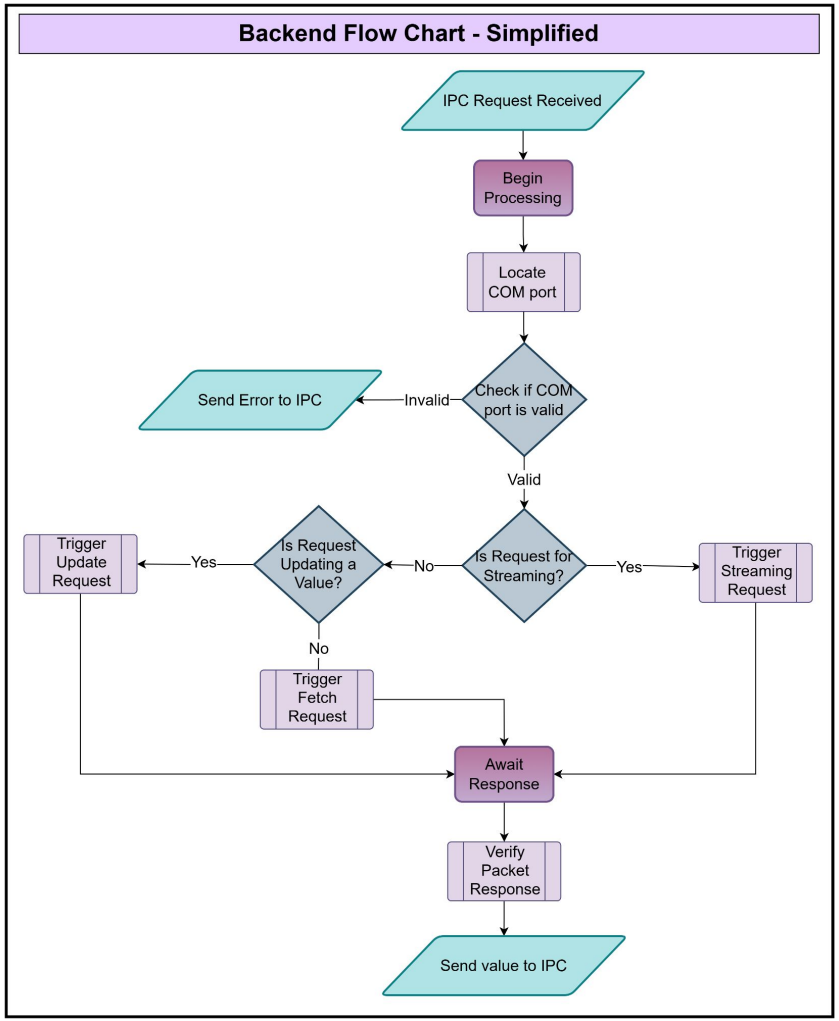
High Level Design



Software - USB

- Backend receives request from IPC
- Backend determines type of request, formats and sends to firmware
- Once response is received, it formats response and sends back to UI through IPC





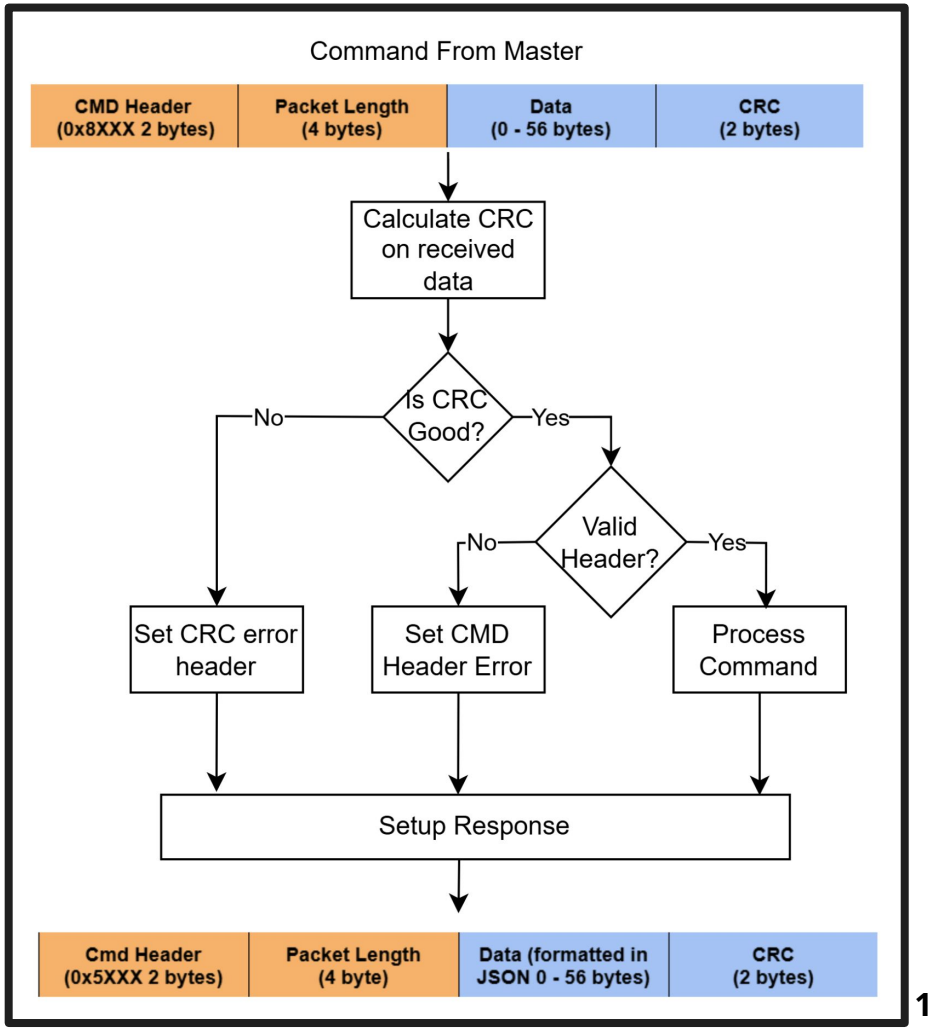
Firmware - USB

- Interrupts trigger processing
- Data read from FIFOs
- 64 byte max packet
- Zero Length Packet signifies end of transmission

Key

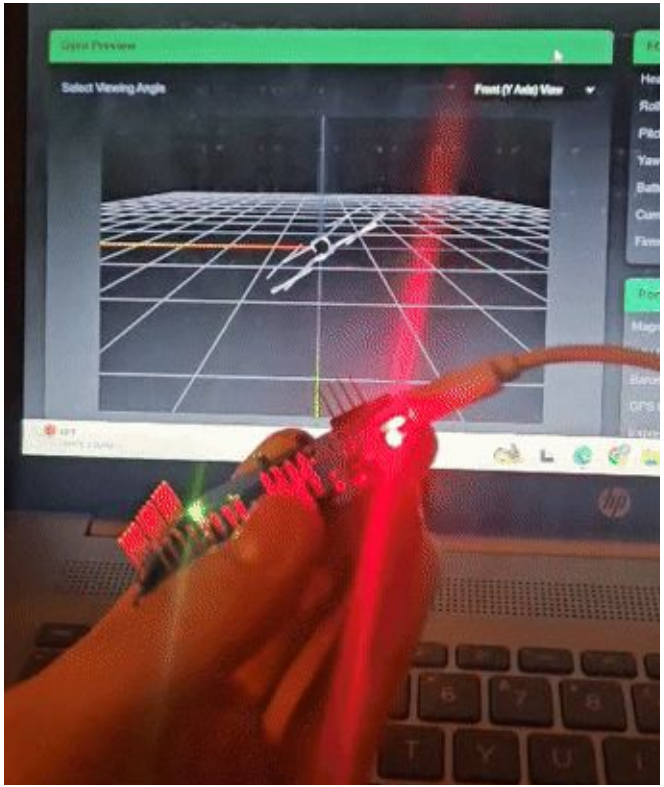
- Decision

- Action/Execution



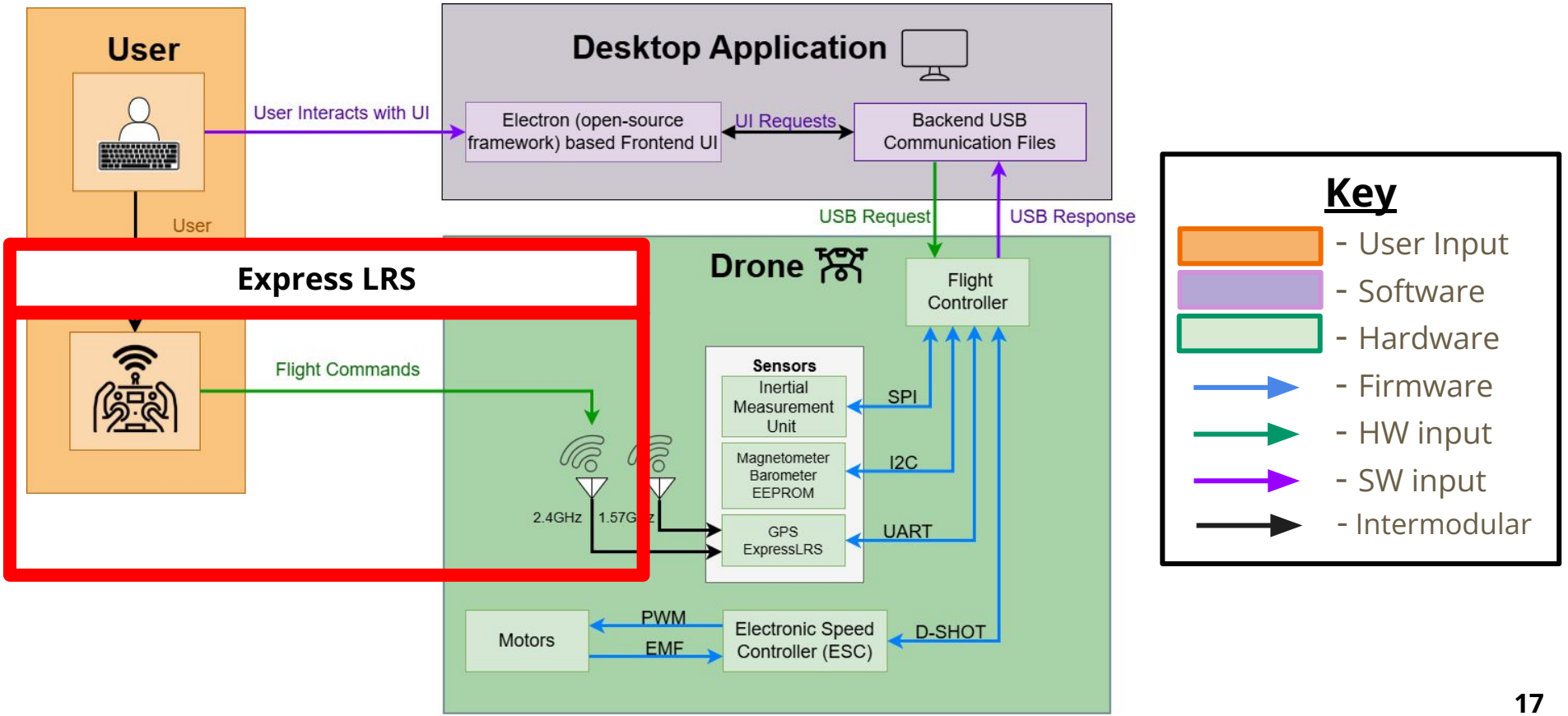
Firmware - USB Streaming

- TinyUSB Driver
- Two COM ports
  - One for streaming, other for commands
- Separate handshakes
- Same packet structure



Gyro Preview

High Level Design

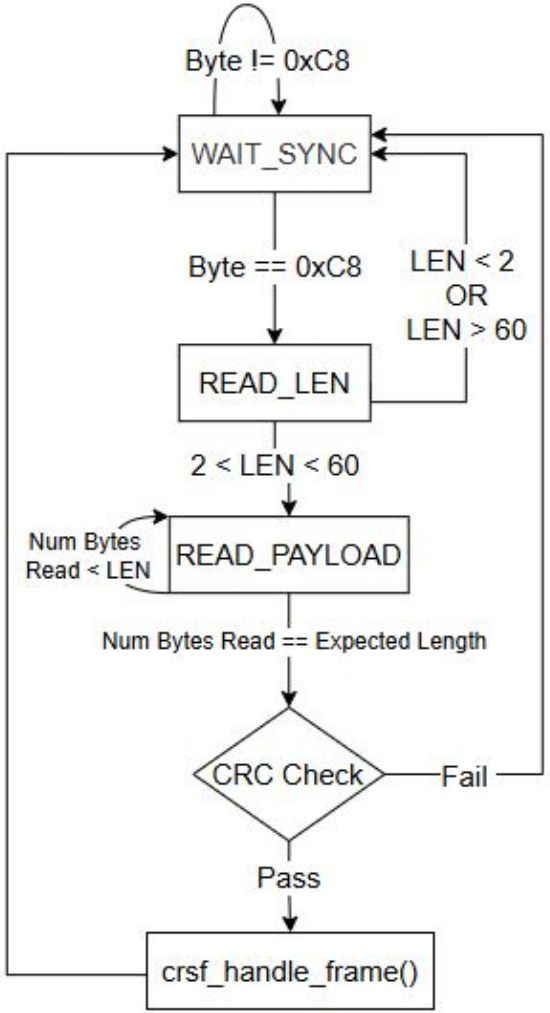


ExpressLRS Protocol

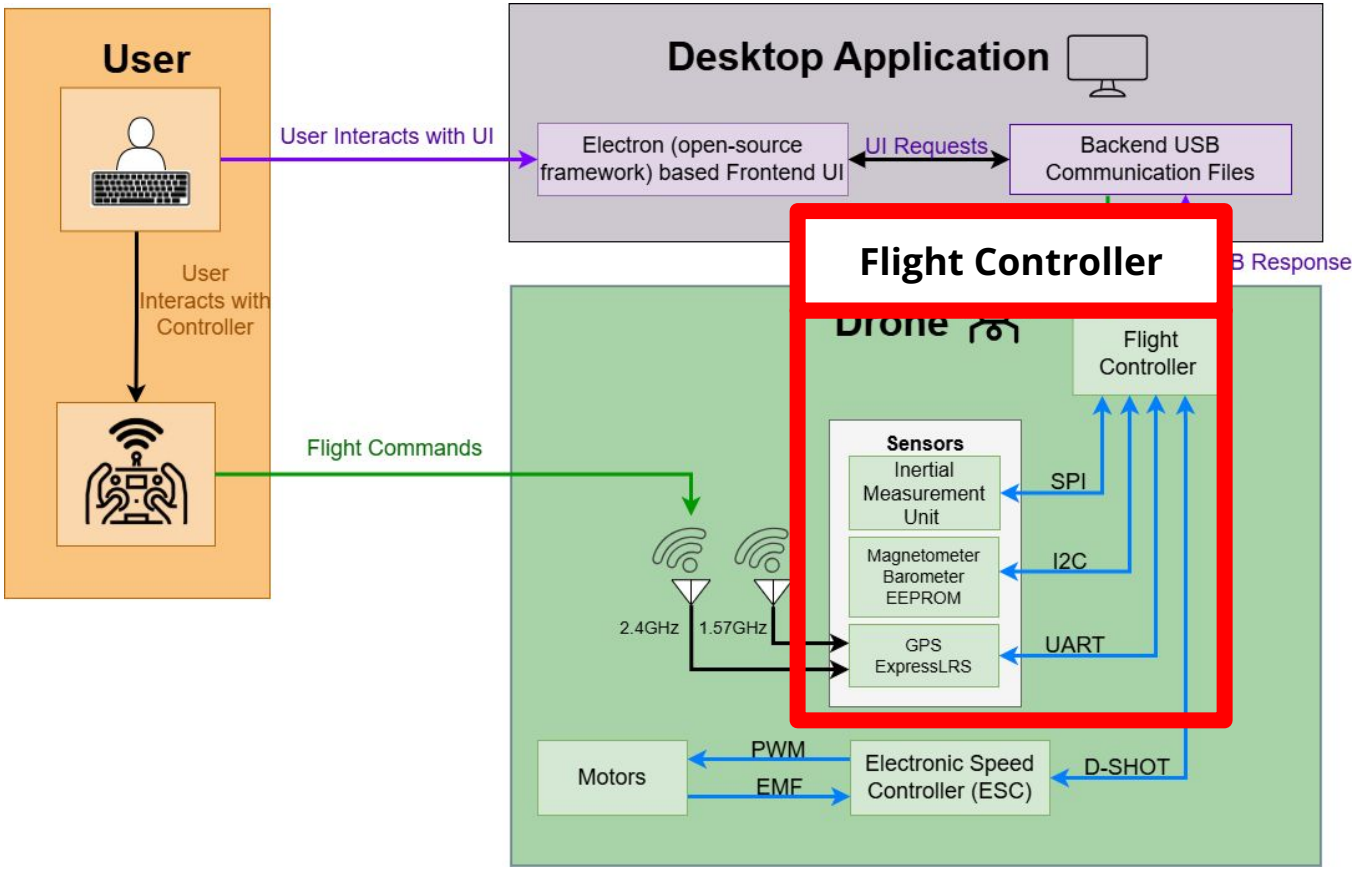
- Off the Shelf EP1 Receiver
- 420,000 Baud Rate
- Receive Data into Direct Memory

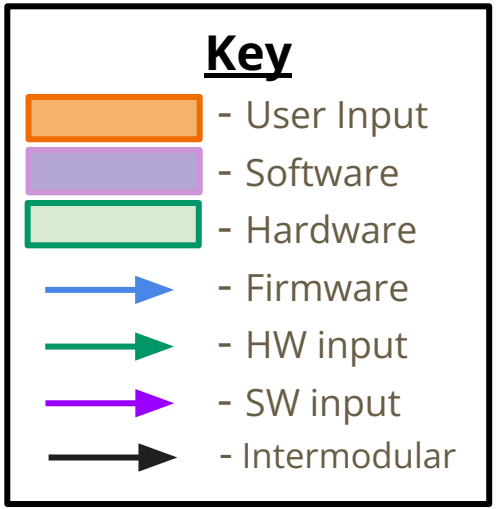


EP1 Receiver



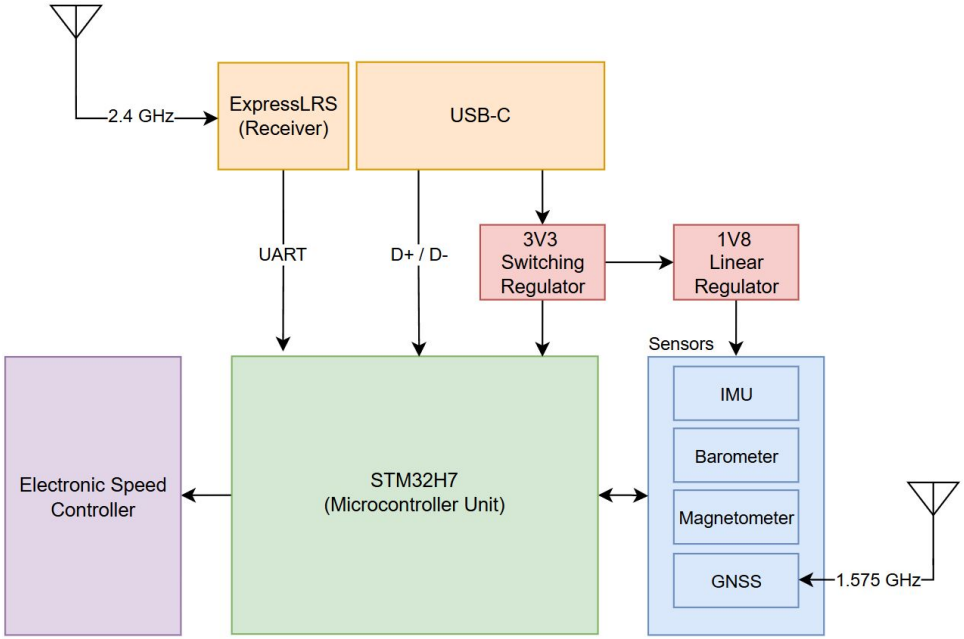
High Level Design





Hardware - Flight Controller

- MCU: *STM32H753VIT6*
  - Clock Rate: 480MHz Cortex-M7, Double-precision FPU
  - I/O: 82 total GPIO (6 SPIs, 4 UARTS, 4 I2C, 22 timers)
- IMU: *ICM-40609-D*
  - Output Data Rate: Up to 32kHz for both accelerometer and gyroscope
  - Accel and Gyro Sensitivity Error: ±0.5%,



Flight Controller High Level

Firmware - FREERTOS (Real-Time Operating System)

- Implemented real-time operating system
- Used tasks, semaphores, mutexes, etc.
- Incorporated to balance timings

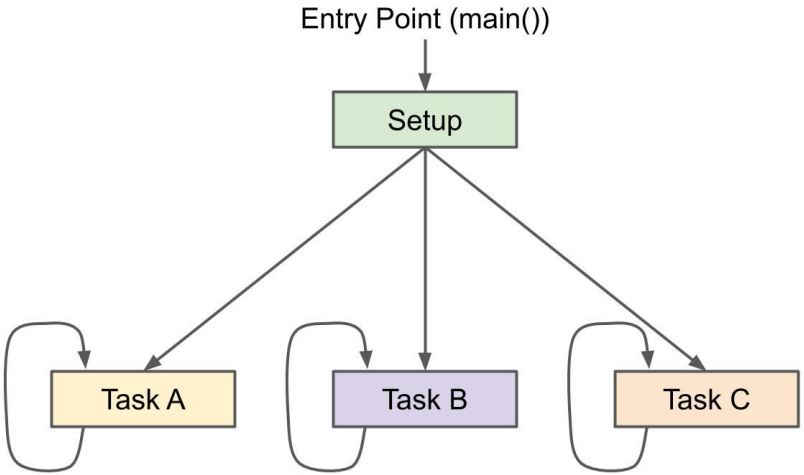
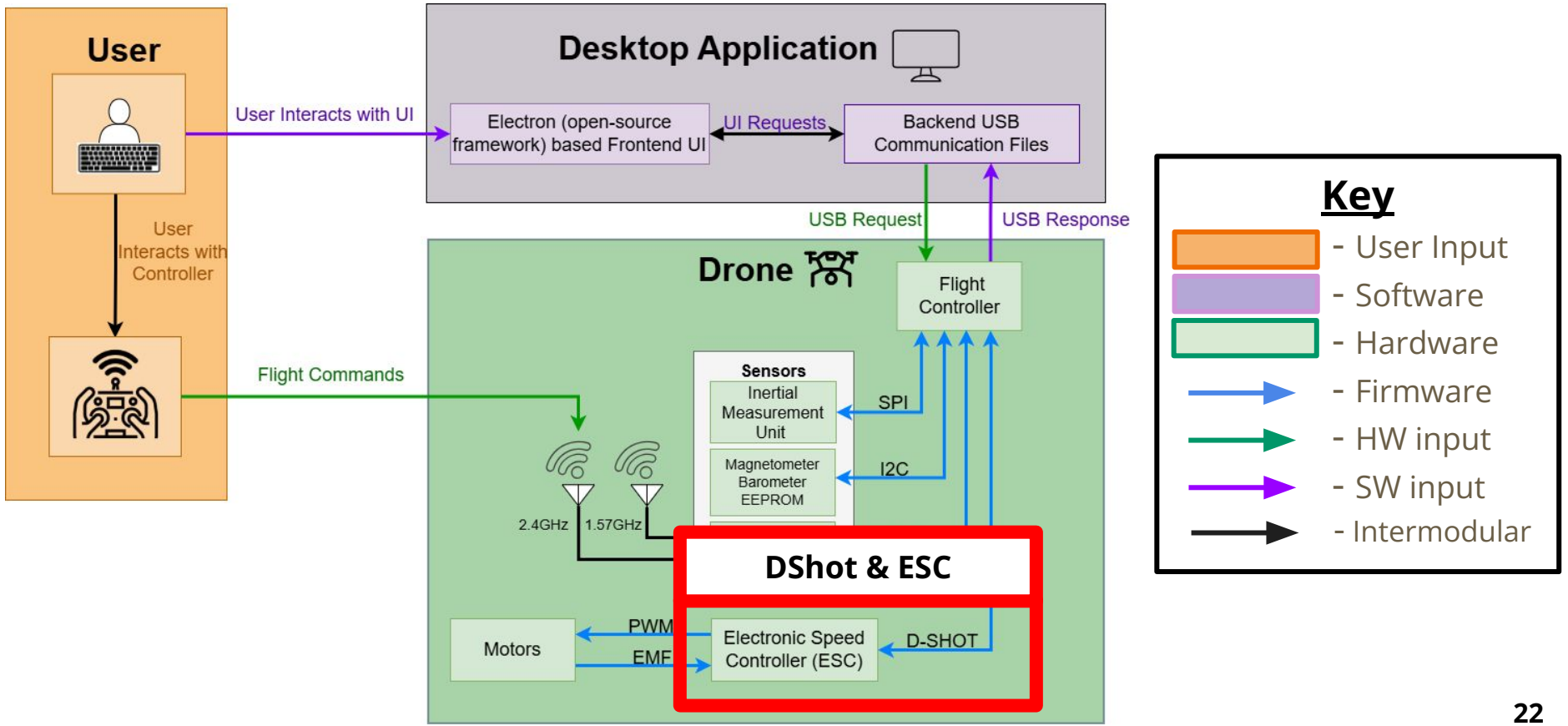


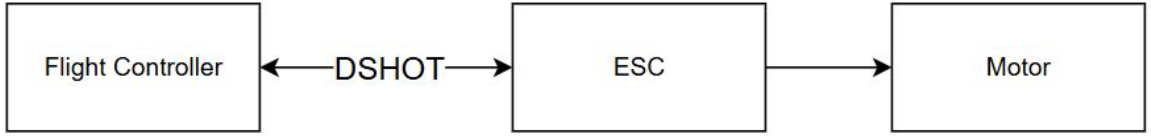
Image from https://www.digikey.com/en/maker/projects/introduction-to-rtos-solution-to-part-3-task-scheduling/8fbb9e0b0eed4279a2dd698f02ce125f

High Level Design

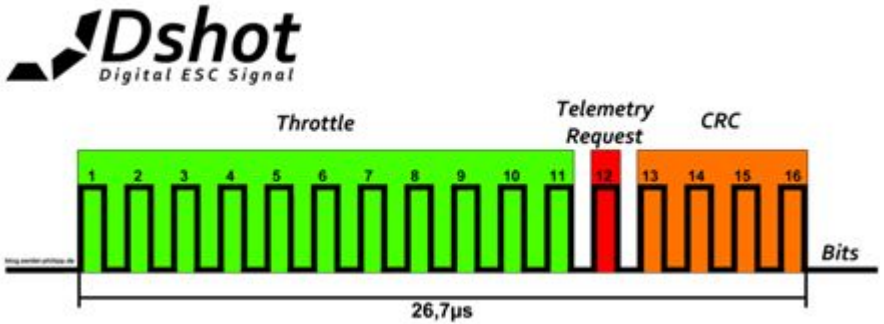


Firmware - DShot (Electronic Speed Controller Protocol)

- Industry standard
- Digital protocol: less susceptible to noise
- Second fastest and most reliable ESC protocol
- Doesn't require calibration



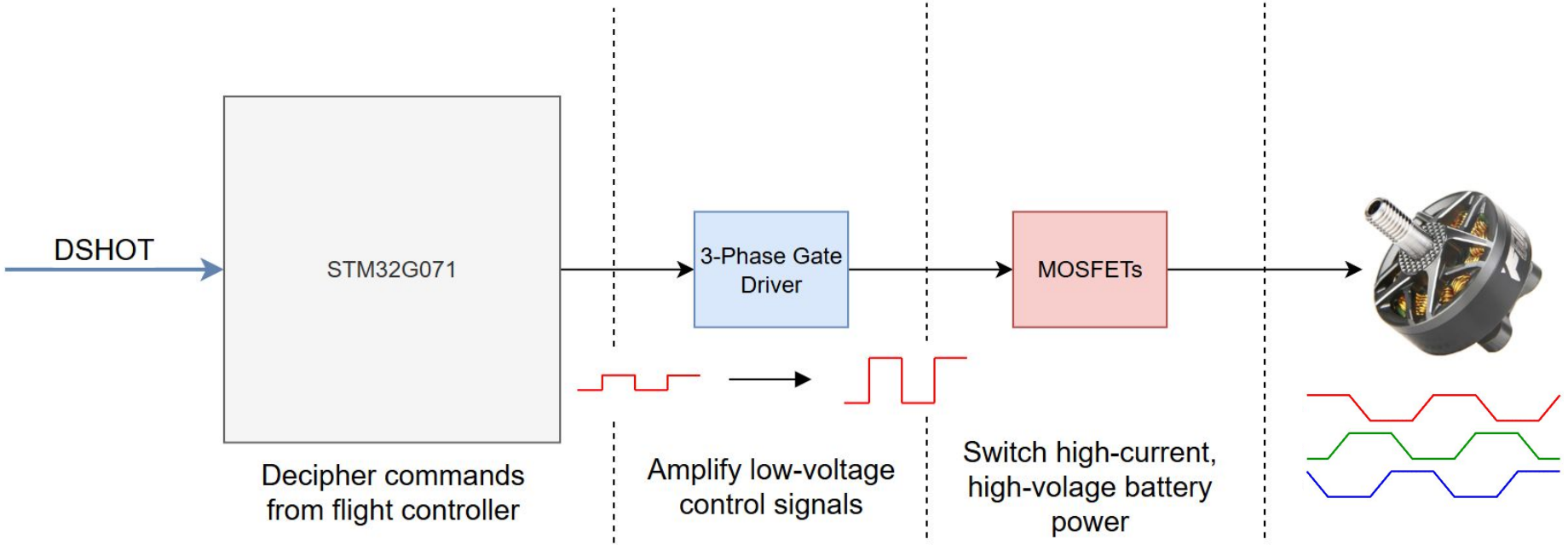
Protocol Motor Stack Position



DShot Frame

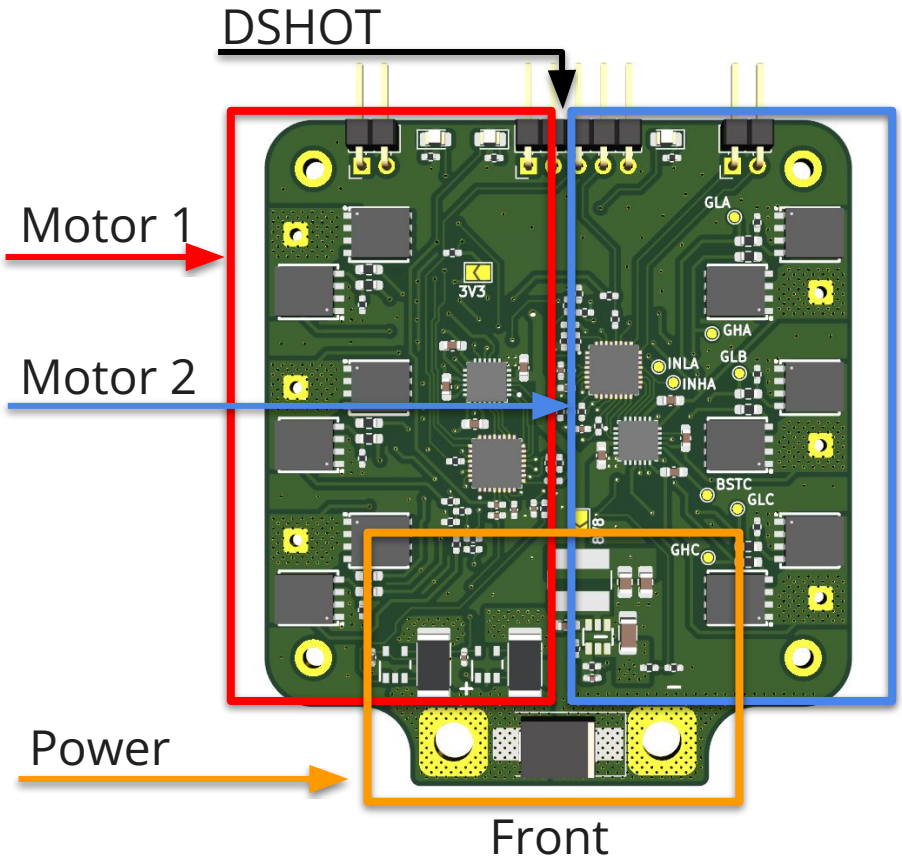
Image from https://www.betaflight.com/docs/development/API/Dshot#:~:text=The%20checksum%20is%20calculated%20over,1000001011000110

ESC Hardware Overview



ESC Design Choices

- 50x50 mm
- Split board
- Current capacity (50A)
  - 2oz copper
  - Large pours
  - T_{MAX} = 115°C



Testing

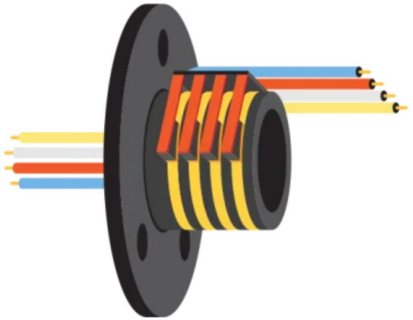
Hardware

Test Setup (Controls Lab)

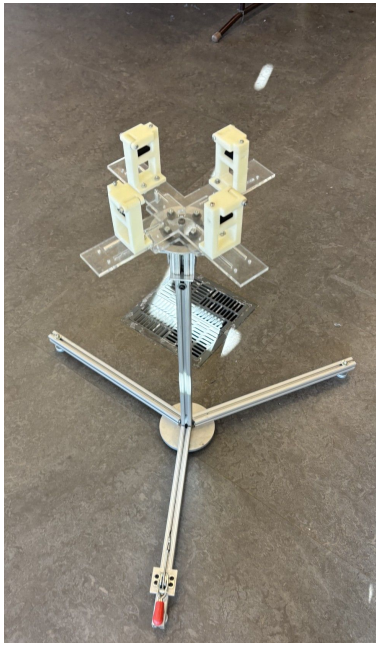
- Constrained frame for safe testing
- Tune roll, pitch, yaw responses
- Conducted in Dr. Jones lab

Slip Ring Integration

- Enables continuous rotation without wires twisting
- Integrated kill switch at rotation origin



Slip Ring



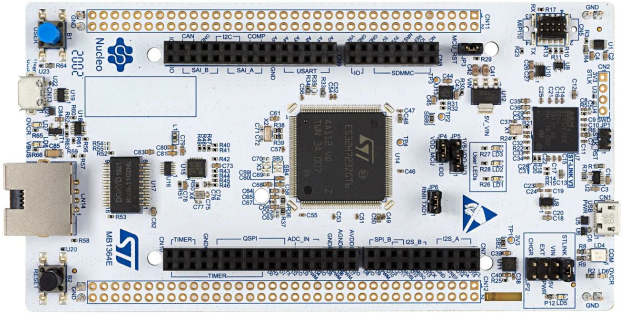
Existing Test Stand in Dr. Jones Controls Lab

Firmware

Unit Testing

Breakout Boards

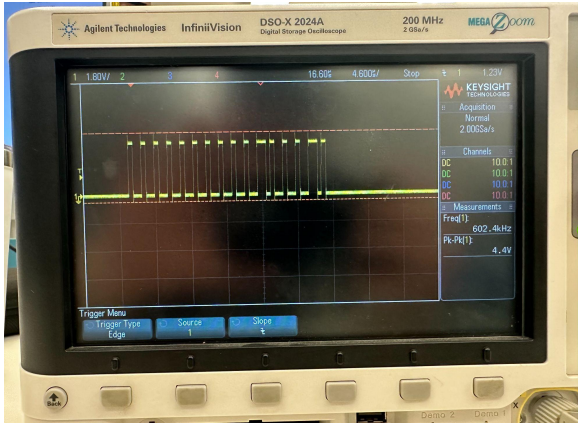
- Known functional hardware
- Test code runnability



H7 Nucleo Breakout Board

Oscilloscope

- Measured communication signals for correct frequency and period



Oscilloscope trace depicting a DShot frame

Integration Testing

HTerm

- COM port watching program to verify device signals
- Manually inspect packets to ensure accuracy

Software

Integration Testing

- Verify communication between the Desktop Application and Firmware

Regression Testing

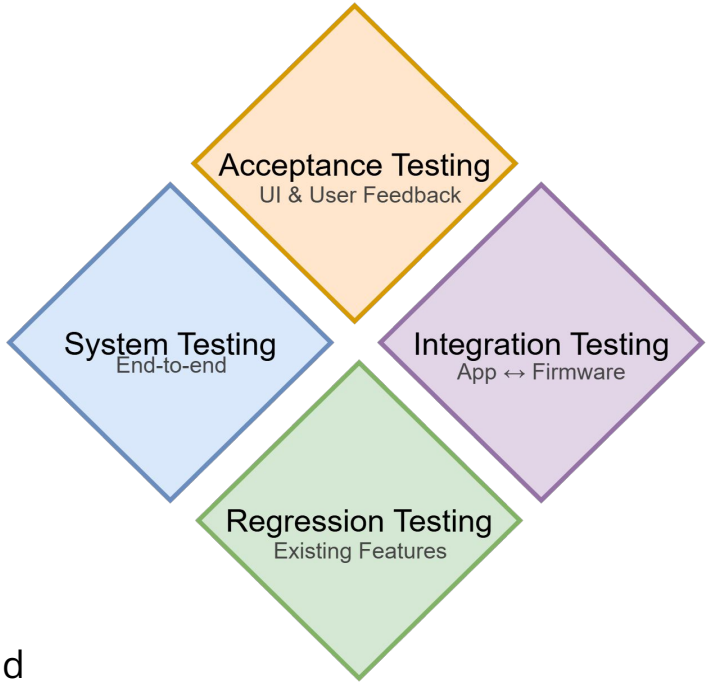
- Ensure new features do not break existing functionality

Acceptance Testing

- Gather user feedback on UI usability and drone configuration controls

System Testing

- Validate end-to-end behavior once Software, Firmware, and Hardware are fully integrated



2nd Semester Design Plan

Workstream	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12	Week 13	Week 14	Week 15
Whole Team	Git Workflow & STMCubeIDE (SOP)	Integration and documentation													Demo week
Hardware	Firmware & Control Assistance		Board revisions			Board Order		New boards (Assembly)	Firmware and Controls Assistance						
Firmware	Getting full motor control			Firmware integration and documentation				Firmware testing							
Software	Replace add dummy data with real data			Get UI feedback on the Desktop Application	Backend for PID Tuning page. Work on GPS and Receiver UI. Make UI updates to the Desktop Application			Updates to documentation and UI on website	Testing						
Controls	Communication with Dr.Jones, Develop Testing procedure, Design test stand		Focus: Controls (Dev boards if ESC fails)					PID tuning testing							

Minimum Viable Deliverable: System with a predictable and stable response to user inputs

Questions?

2nd Semester Plan - Key Phases and Deliverables

- **Phase 1: Integration & Baseline Control (Weeks 1-4)**
  - SW: testing actual data instead of dummy data
  - FW: Establishing stable motor control and validating sensor data
  - HW: Assisting FW and confirming board reliability
- **Phase 2: System Build & Feature Expansion (Weeks 5-9)**
  - SW: Builds out backend for PID tuning and continues UI development
  - FW: Full integration and documentation
  - HW: Orders new board and assembles if necessary
- **Phase 3: Testing & Validation (Weeks 10-14)**
  - Testing and debugging on all teams with a focus on PID tuning and validation
- **Phase 4: Demo Readiness (Week 15)**
  - Checking that the system meets the minimum requirement of a fully integrated, stable system that responds reliably to user inputs.

2nd Semester Plan

Phase 1: Baseline
(Weeks 1-4)

- Motor Control & sensor Validation
- Replace Dummy data with real data

Deliverable:
Stable, predictable response to user input

Phase 2: System Build
(Weeks 5-9)

- Firmware Integration & Documentation
- PID tuning backend and UI updates
- Board Revisions & assembly

Deliverable:
Fully integrated system

Phase 3: Testing & Tuning
(Weeks 10-14)

- Firmware & Software testing
- PID tuning and validation
- Controls Robustness

Deliverable:
Reliable and tuned system performance

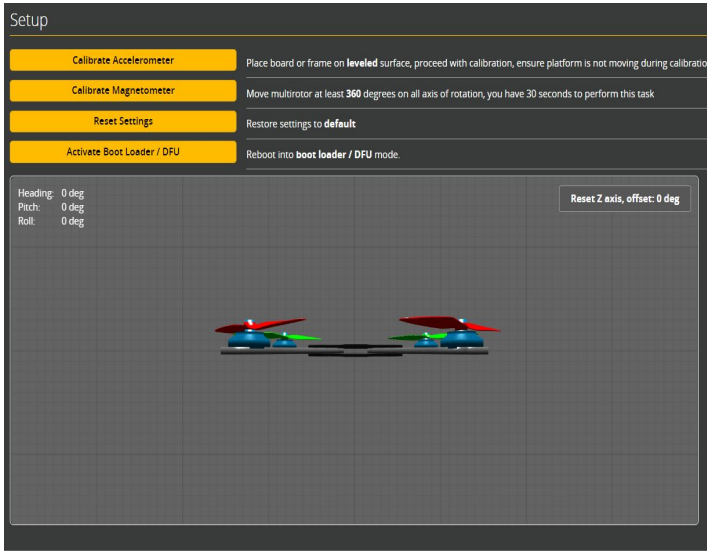
Phase 4: Demo Readiness
(Week 15)

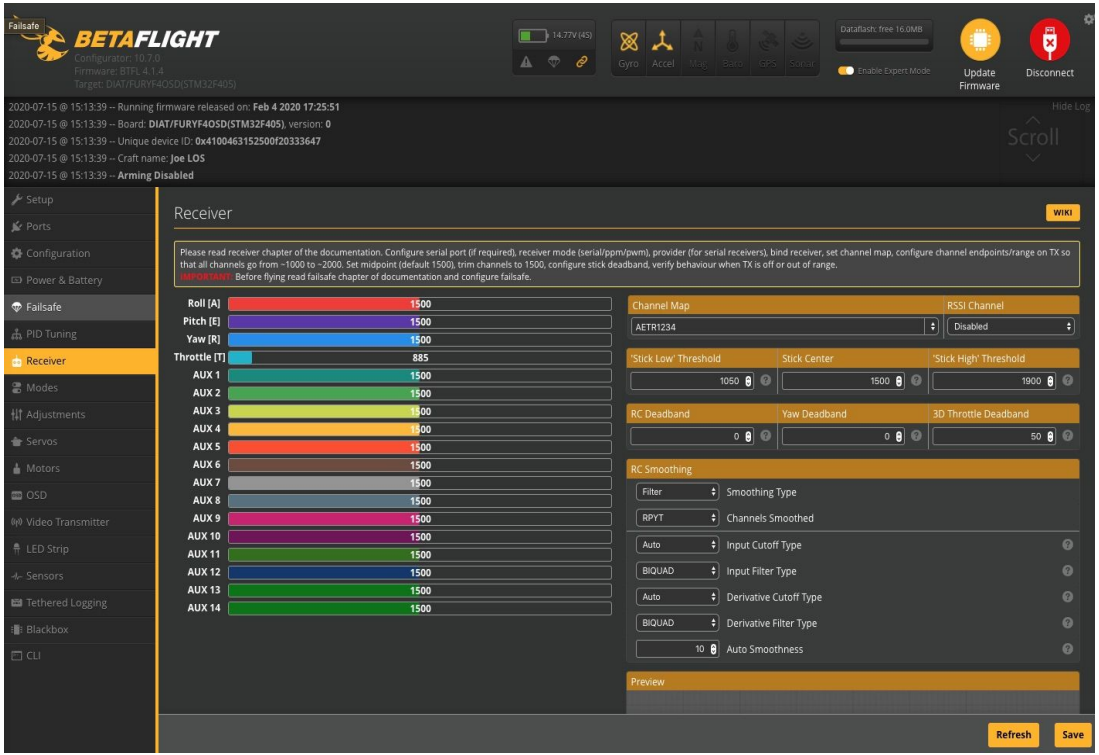
- Final System validation
- Integration Polish

Deliverable:
Final demo-ready system

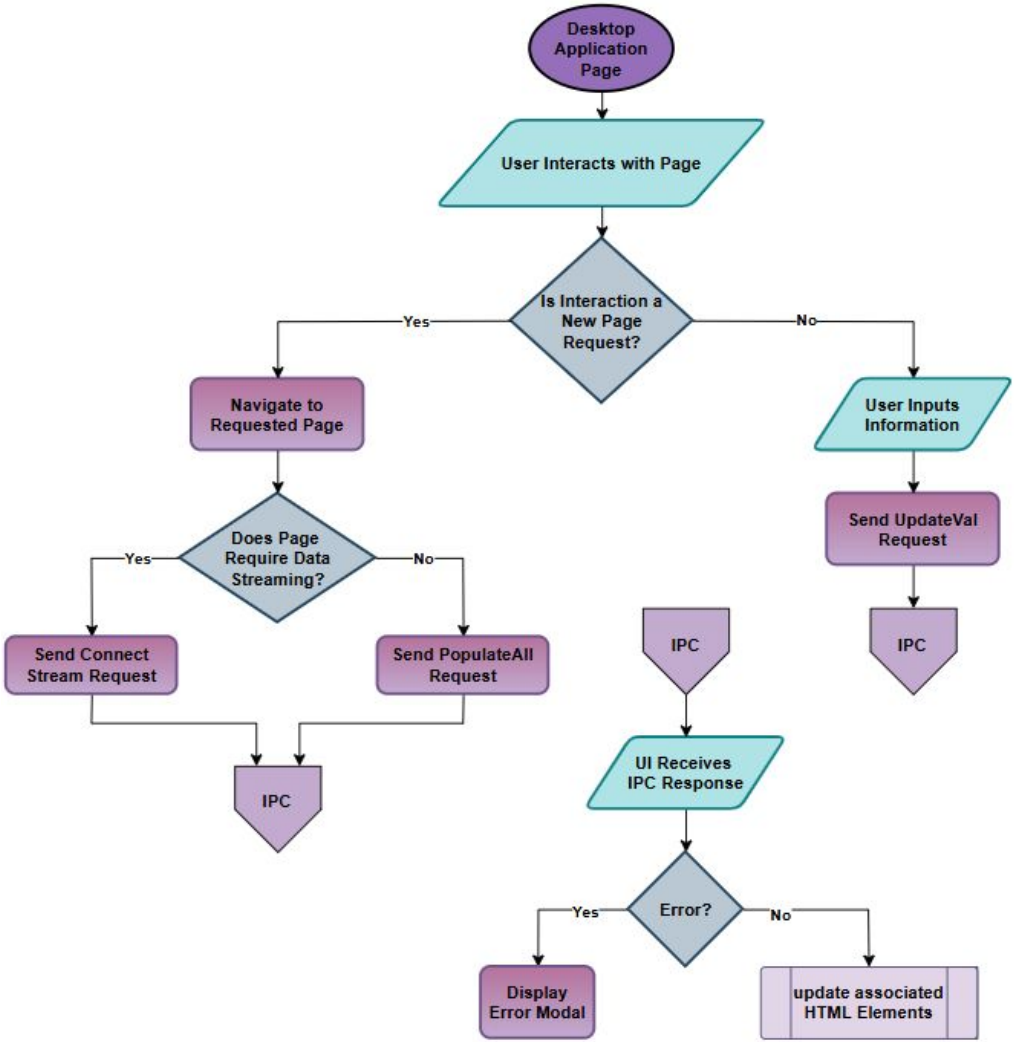
Software Appendix

Betaflight


Setup
Calibrate Accelerometer
Place board or frame on leveled surface, proceed with calibration, ensure platform is not moving during calibratio
Calibrate Magnetometer
Move multirotor at least 360 degrees on all axis of rotation, you have 30 seconds to perform this task
Reset Settings
Restore settings to default
Activate Boot Loader / DFU
Reboot into boot loader / DFU mode.
Heading: 0 deg
Pitch: 0 deg
Roll: 0 deg
Reset Z axis, offset: 0 deg

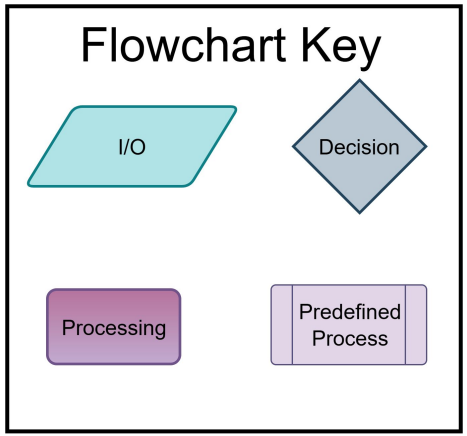

BETAFLIGHT
Update Firmware
Disconnect
Setup
Ports
Configuration
Power & Battery
Failsafe
PID Tuning
Receiver
Modes
Adjustments
Servos
Motors
OSD
Video Transmitter
LED Strip
Sensors
Tethered Logging
Blackbox
CLI
Receiver
WIKI
Roll [A] 1500
Pitch [E] 1500
Yaw [R] 1500
Throttle [T] 885
AUX 1 1500
AUX 2 1500
AUX 3 1500
AUX 4 1500
AUX 5 1500
AUX 6 1500
AUX 7 1500
AUX 8 1500
AUX 9 1500
AUX 10 1500
AUX 11 1500
AUX 12 1500
AUX 13 1500
AUX 14 1500
Channel Map
RSSI Channel
AETR1234
Disabled
'Stick Low' Threshold
Stick Center
'Stick High' Threshold
RC Deadband
Yaw Deadband
3D Throttle Deadband
RC Smoothing
Smoothing Type
Channels Smoothed
Input Cutoff Type
Input Filter Type
Derivative Cutoff Type
Derivative Filter Type
Auto Smoothness
Preview
Refresh
Save

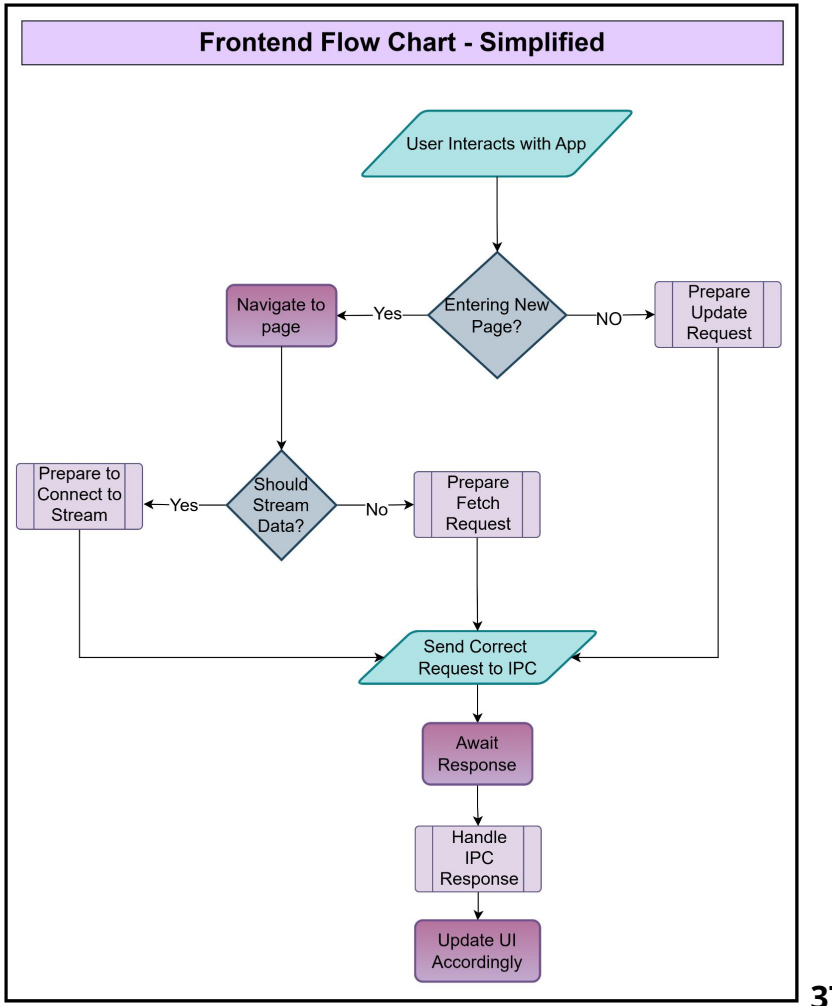
Detailed Frontend Flowchart



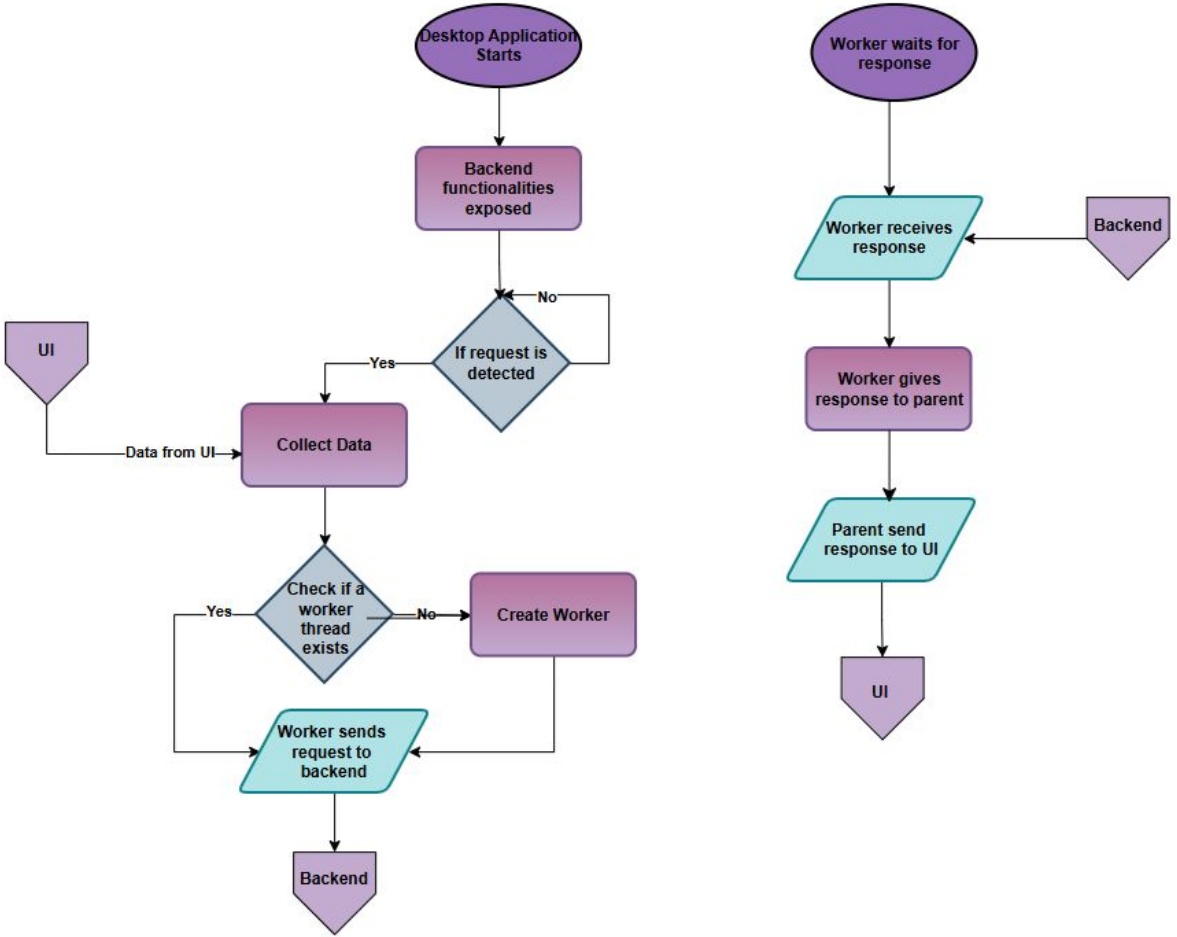
Desktop Application - Frontend

- User interacts with the UI to trigger requests to be sent
- Requests are then processed and sent to the backend
- Once response is received the UI is updated

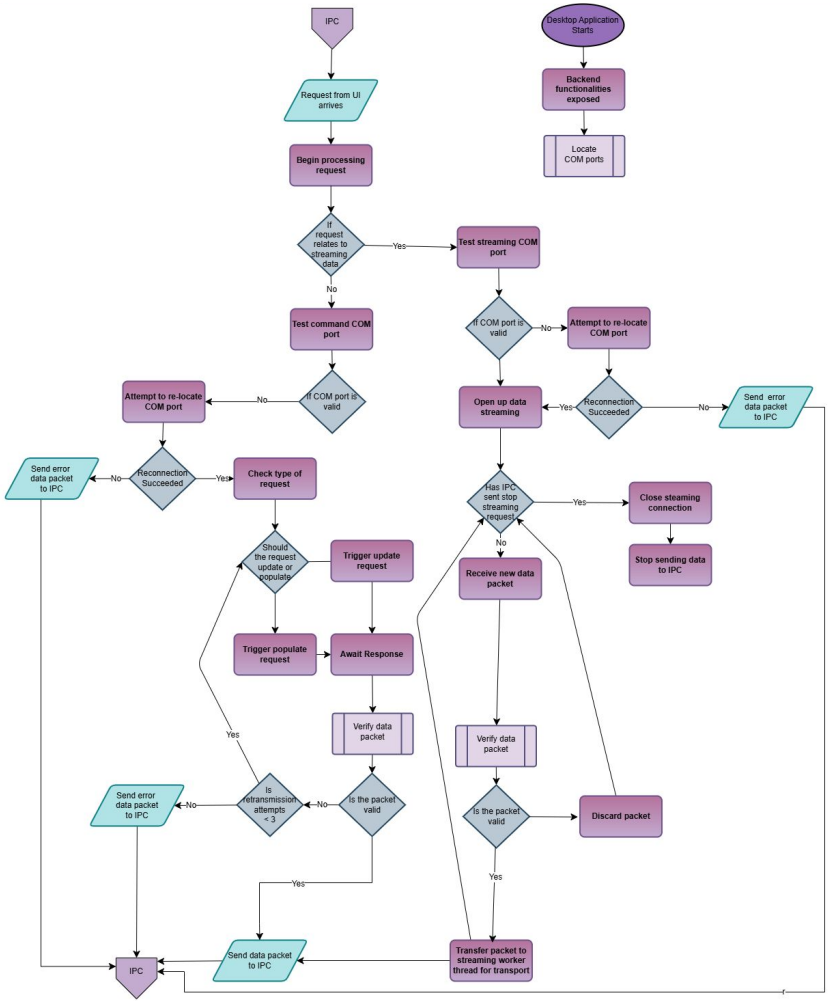




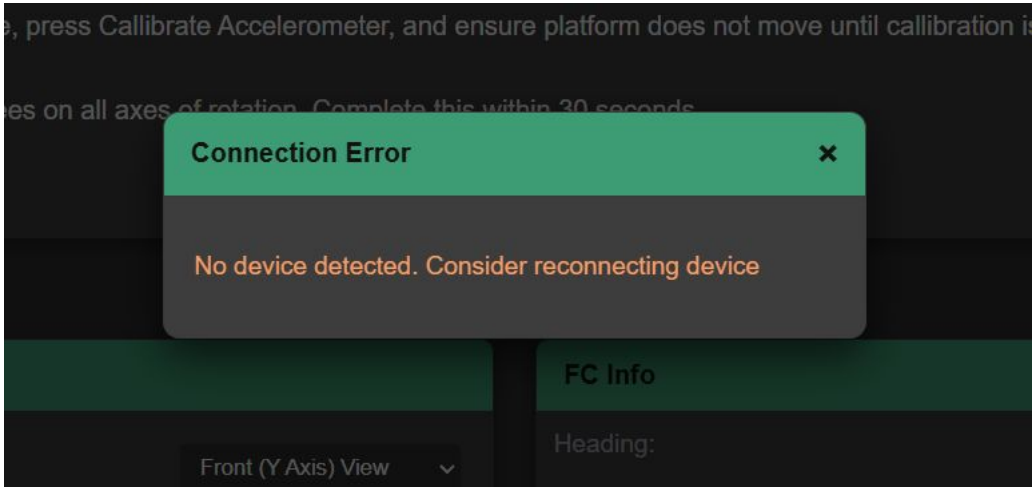
Detailed IPC Flowchart



Detailed Backend Flowchart



Error Handling



Frontend

- Pop-up error triggers from failed IPC responses

Backend

- **Command-Response Request**:
  - CRC mismatch triggers request retry
  - Firmware-signaled CRC error triggers request retry
  - 3 failed retries returns error JSON formatted string to frontend
- **Streaming Request**:
  - Silently drops packet upon CRC check fail and queue overflow.
  - If for some reason the Stop command fails to trigger, the streaming thread exits anyways.
- A NULL value is never returned to the frontend. Only JSON formatted error strings

Node-API

What is it?

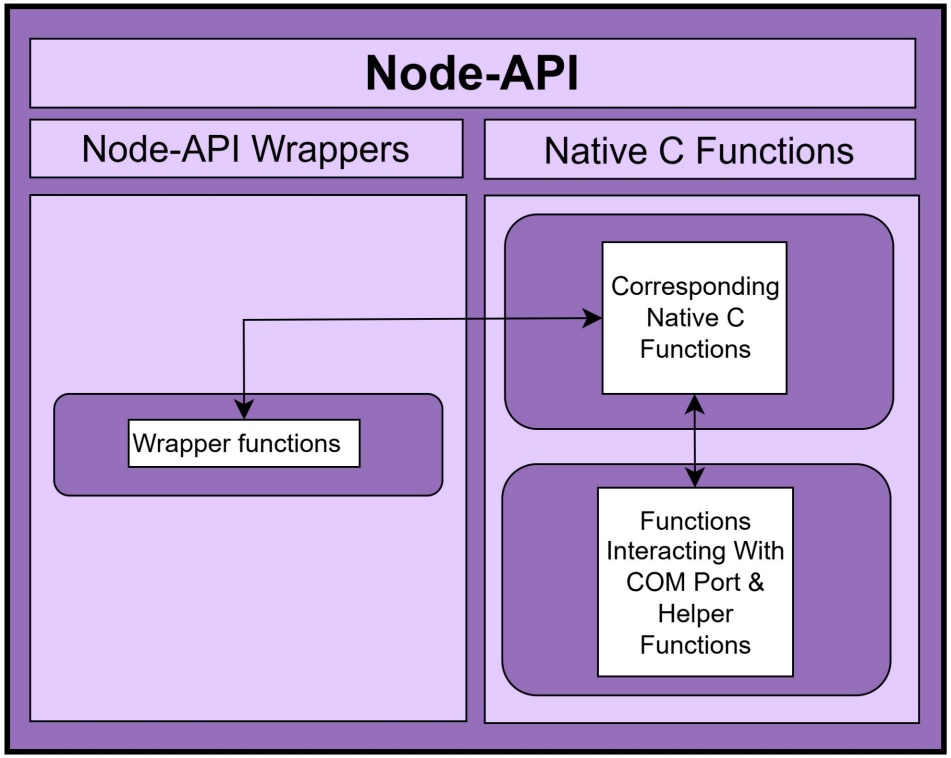
- A stable C interface for building Node.js and Electron file addons

What does it do?

- Wraps C functions so that JavaScript can call them
- Handles JavaScript ↔ C type conversion
  - Extracts UTF-8 strings from JS and returns strings/booleans back to JS
- Macros register the addon as a Node.js module on app load

Why use it?

- Stable across Node/Electron versions
- Allows using the C language within a Node.js environment



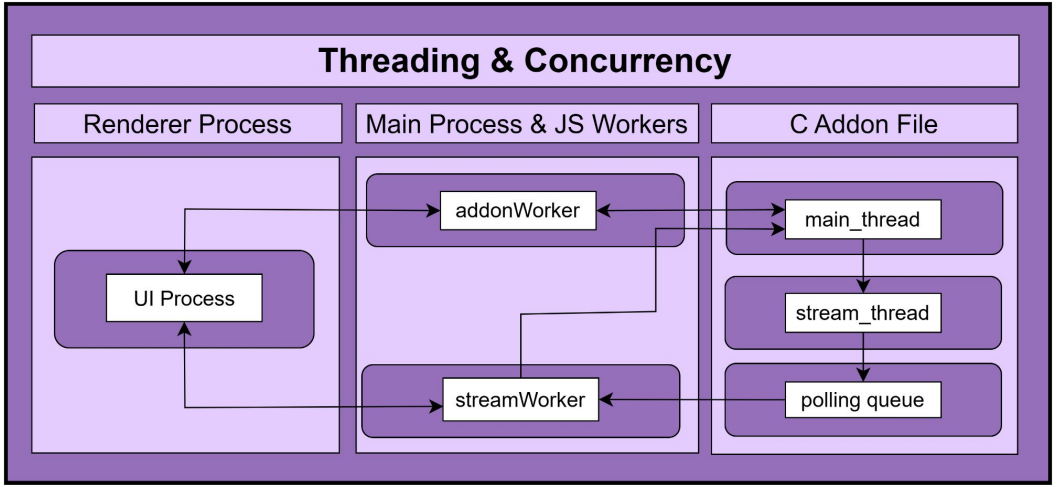
Threading & Concurrency

One C Thread

- Stream_thread controls the stream COM port exclusively:
  - Reads packets in a loop
  - Pushes JSON value of packet to a circular buffer queue
  - Then allows polling from frontend
- Queue access is protected by mutex locks. This prevents race conditions when the C thread is writing to the queue or the JavaScript thread is reading from the queue

Two JavaScript Worker Threads

- addonWorker handles command port calls
- streamWorker handles polling from Node-API exposed function



IPC communication structure

Process Isolation

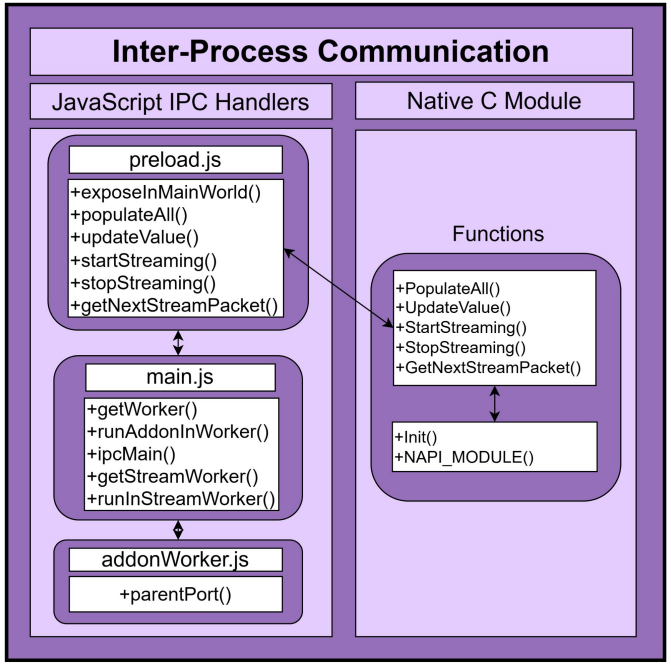
- Electron enforces process isolation
  - Renderer process cannot access backend or the addon directly
  - Main process handles all requests
- One JS file acts as the bridge to the C addon by exposing it to the frontend

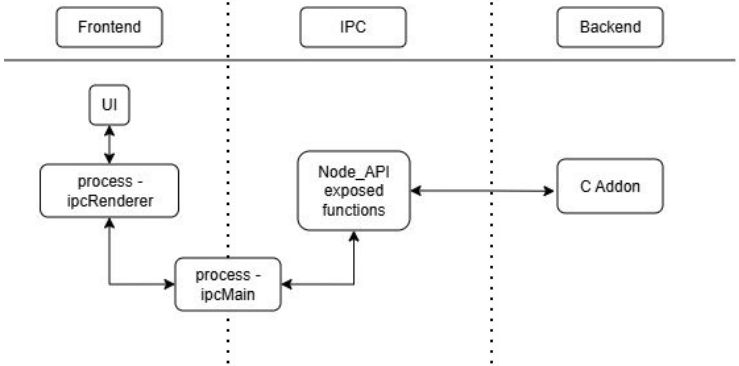
Renderer Process

- Calls C serial functions held in the main process then forwards it to a worker thread for processing
- Handles updates to the UI
- Promise-based so the UI can wait for information without blocking

Main Process

- Streaming: Polls worker and pushes packets to renderer at ~60 fps
- Command-Response: Sends request, awaits response, forwards response to the Renderer

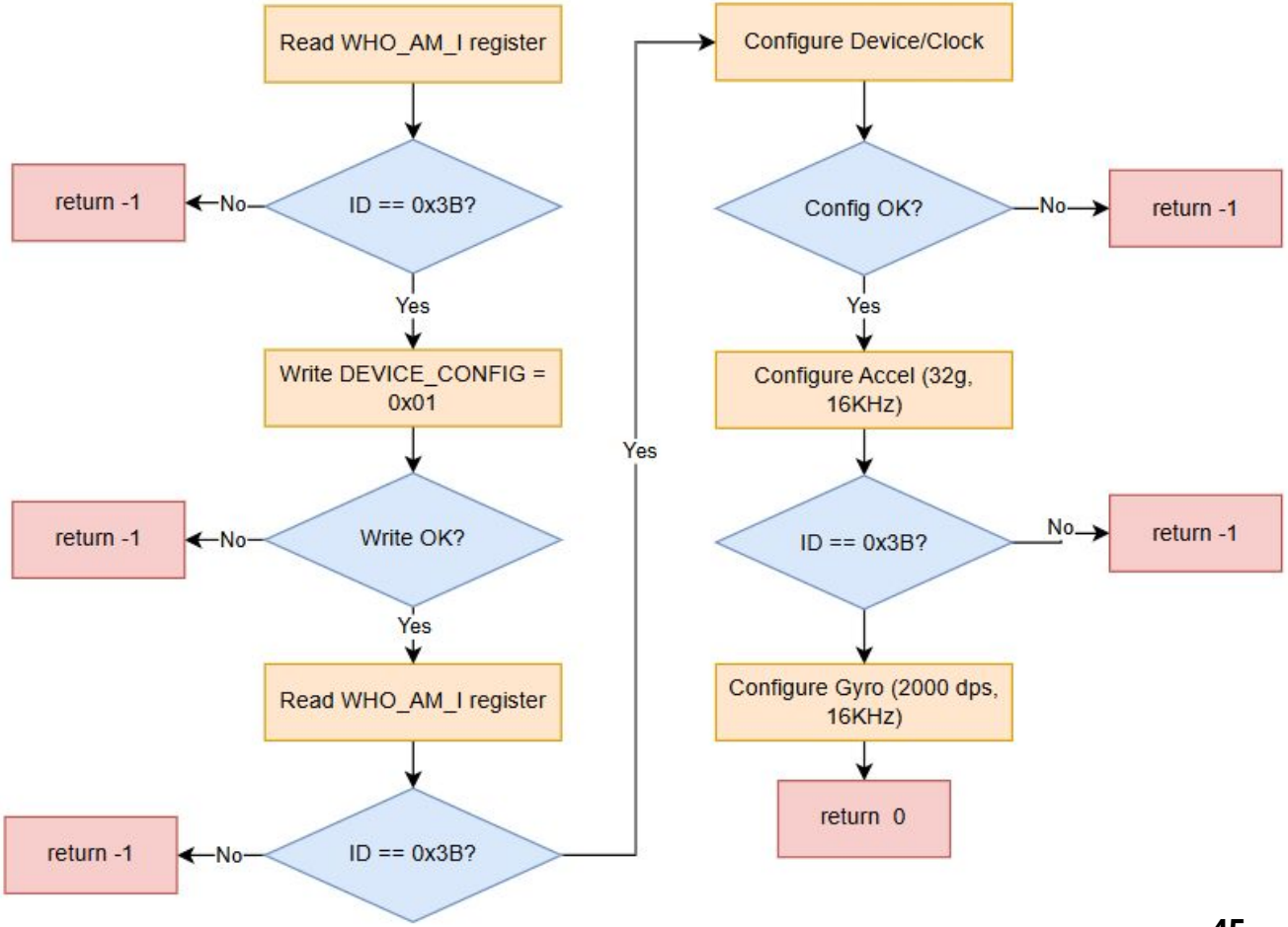




Firmware Appendix

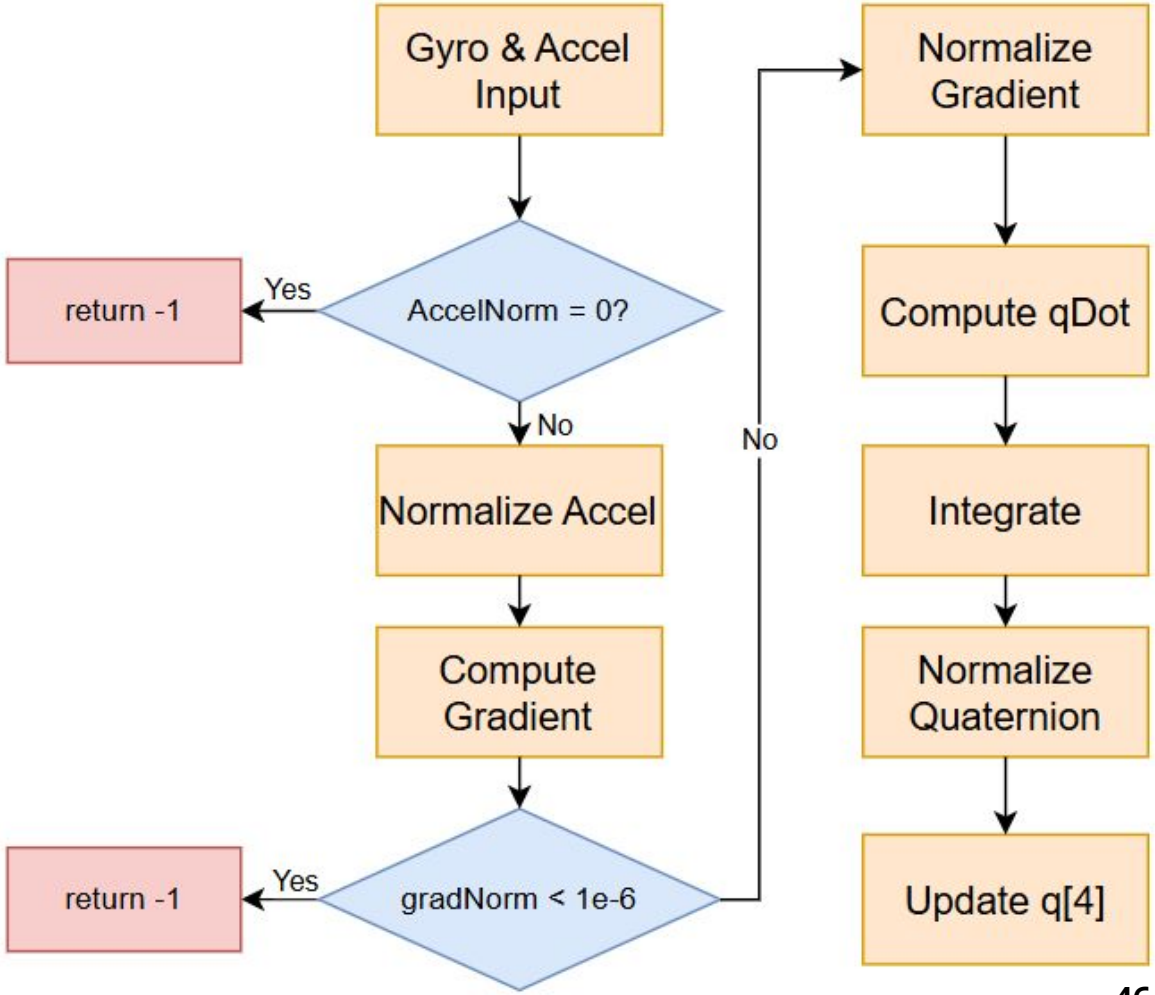
IMU Init

- 2000 degrees/sec
- 32 g
- 32KHz data rate



Madgwick Filter

- Using Accelerometer to help stabilize gyro drift
- Beta 0.25
  - Accel Dominates



Key

◇ - Decision

▭ - Action/Execution

Spherical Linear Interpolation (SLERP)

- Quaternions are in a 4-D Space
- SLERP finds the shortest valid path between 2 orientations
  - While keeping a valid quaternion value

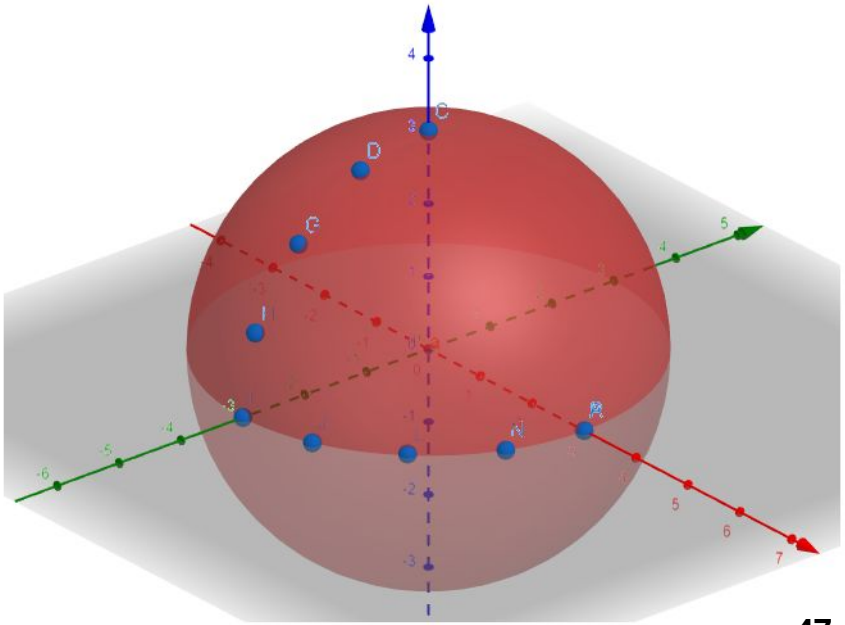
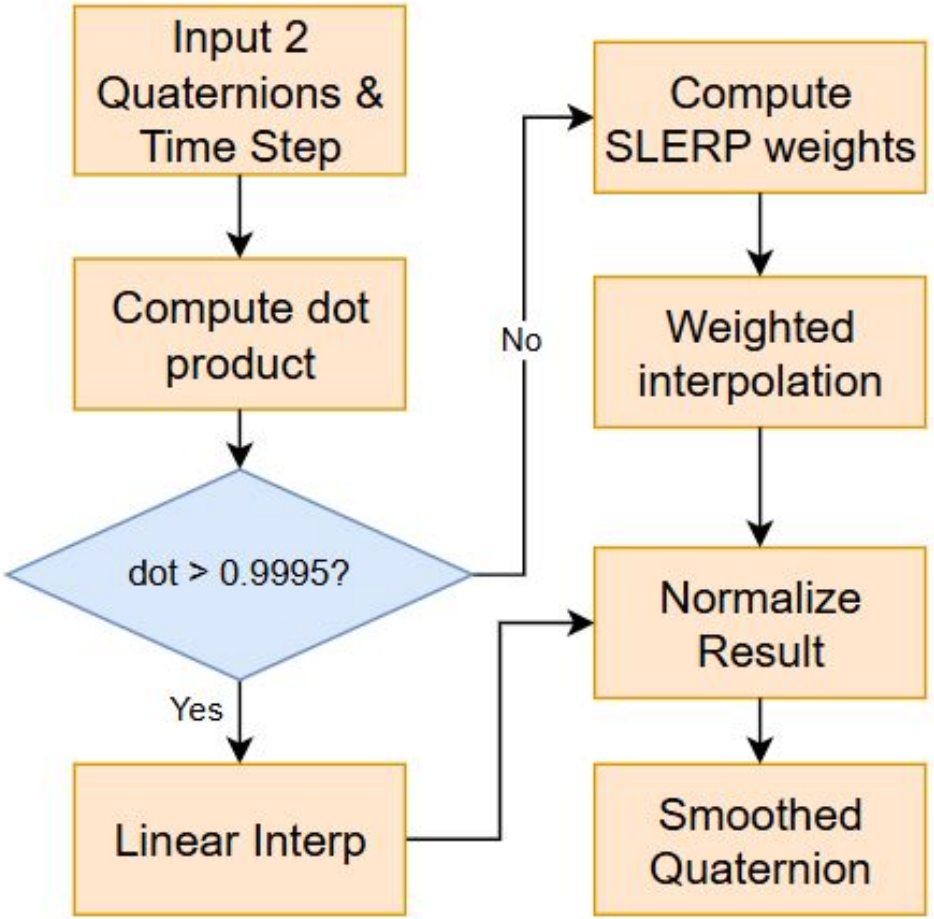


Image from: https://stackoverflow.com/questions/62937791/quaternion-based-point-rotations-using-glm

SLERP Smoothing

- Acts as a smoothing window between quaternions

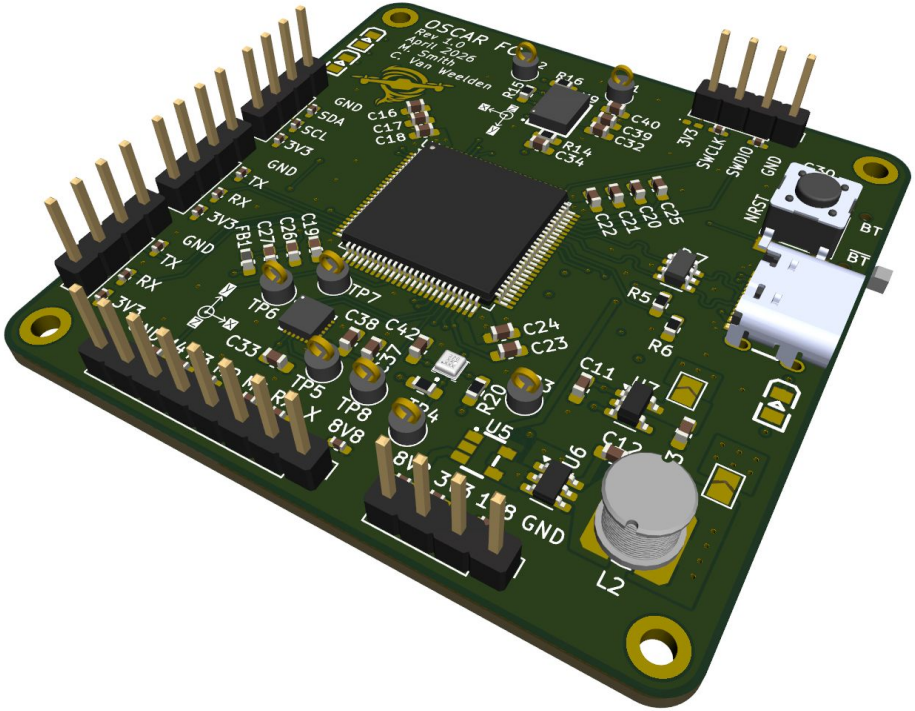


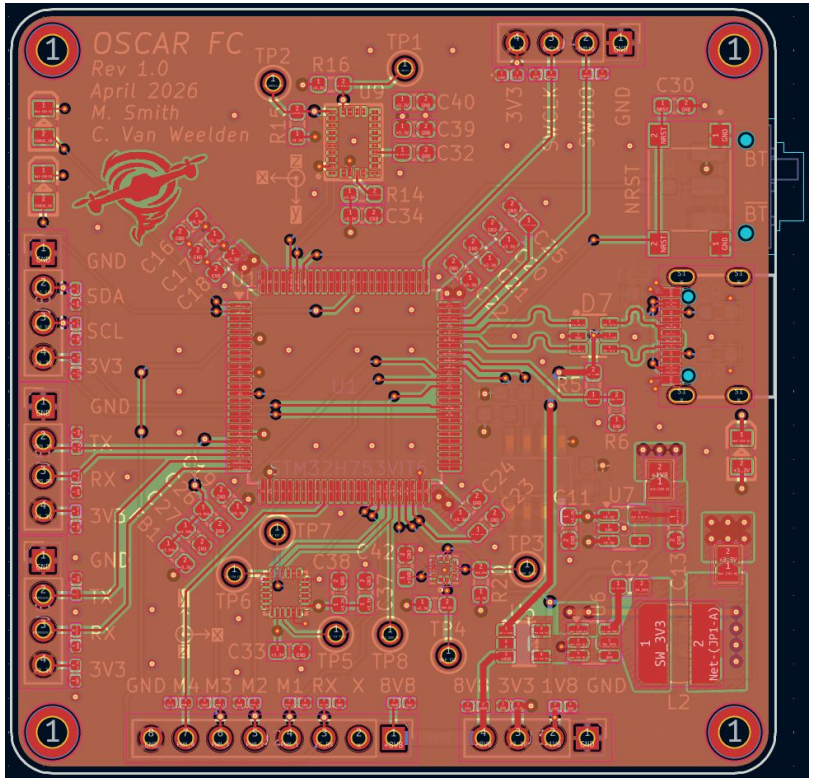
Key

- Decision
- Action/Execution

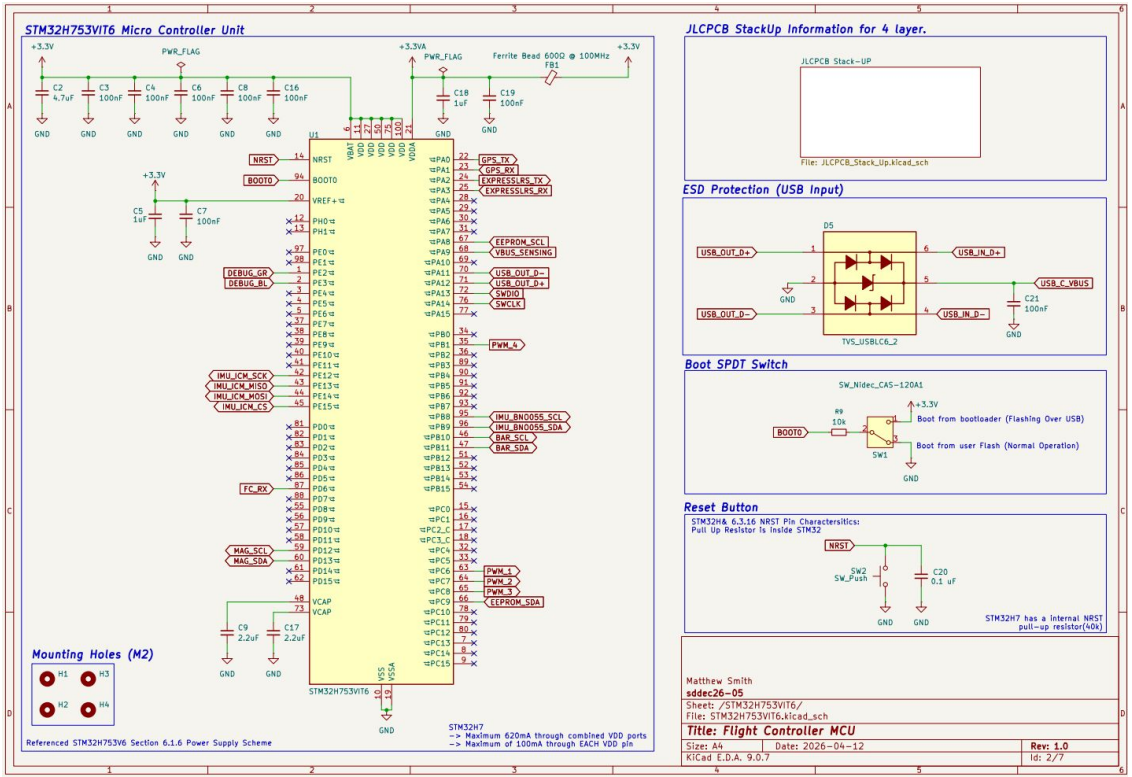
Hardware Appendix

Flight Controller Layout



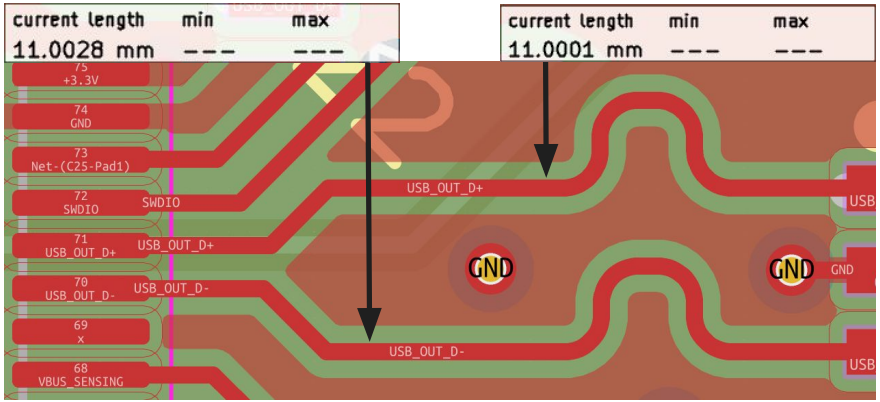


Flight Controller Schematic (MCU)

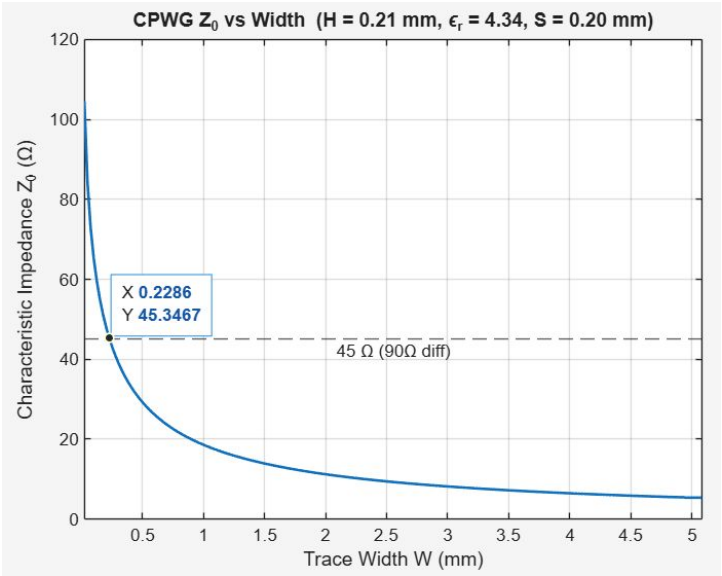


Hardware - USB

- USB Type-C
- USB 2.0 Full Speed (**12Mbps**)
- Differential Signaling
  - Time Matching
  - Impedance Matching

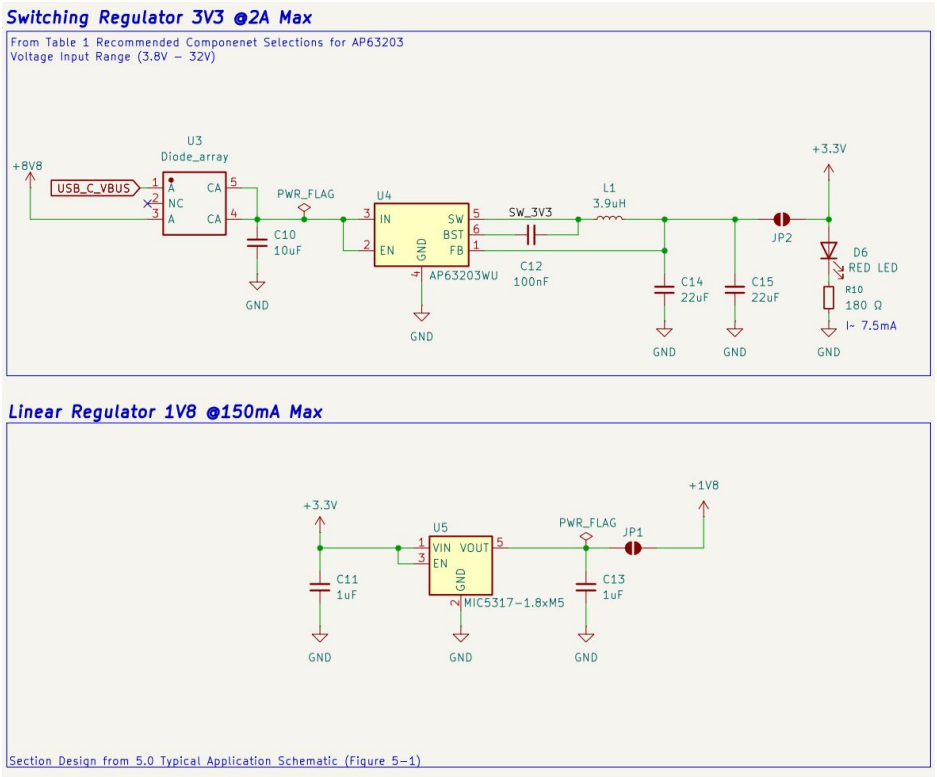


Time Matched

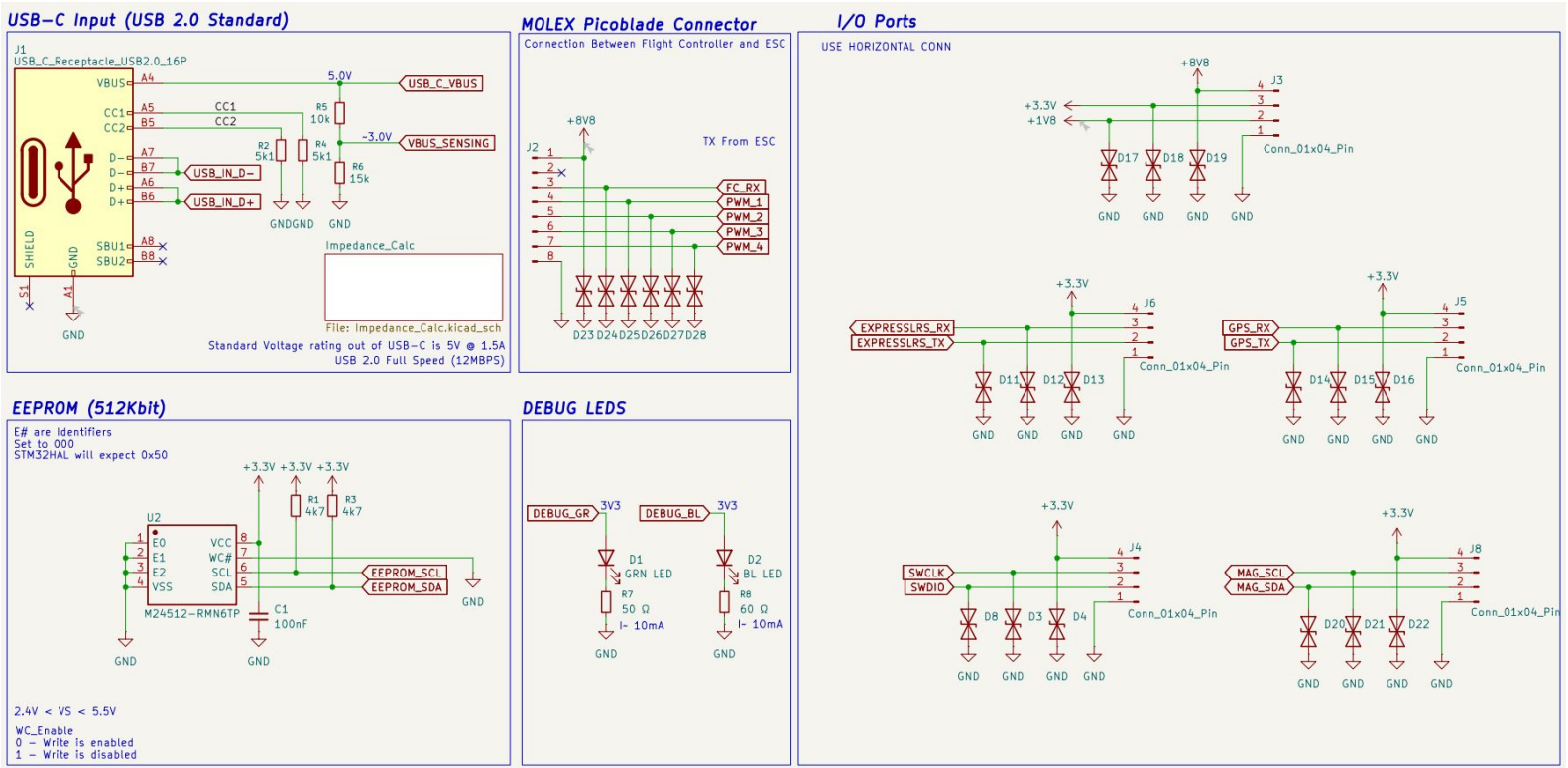


Impedance Matched

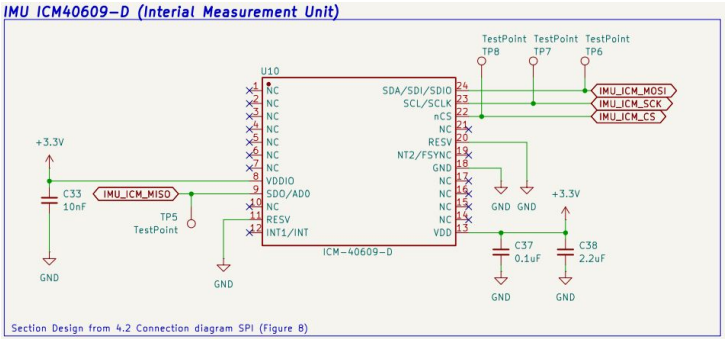
Flight Controller Schematic (Voltage Regulaton)

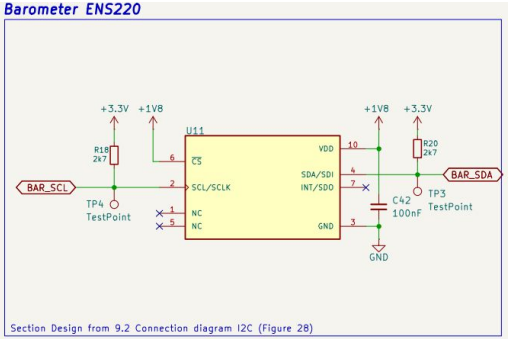


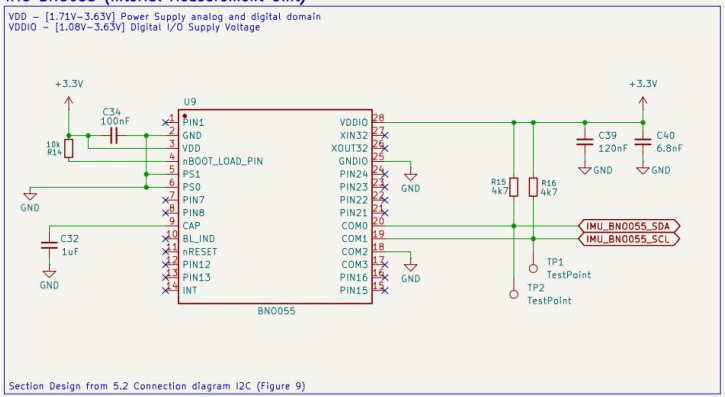
Flight Controller Schematic (Peripherials)



Flight Controller Schematic (Sensors)







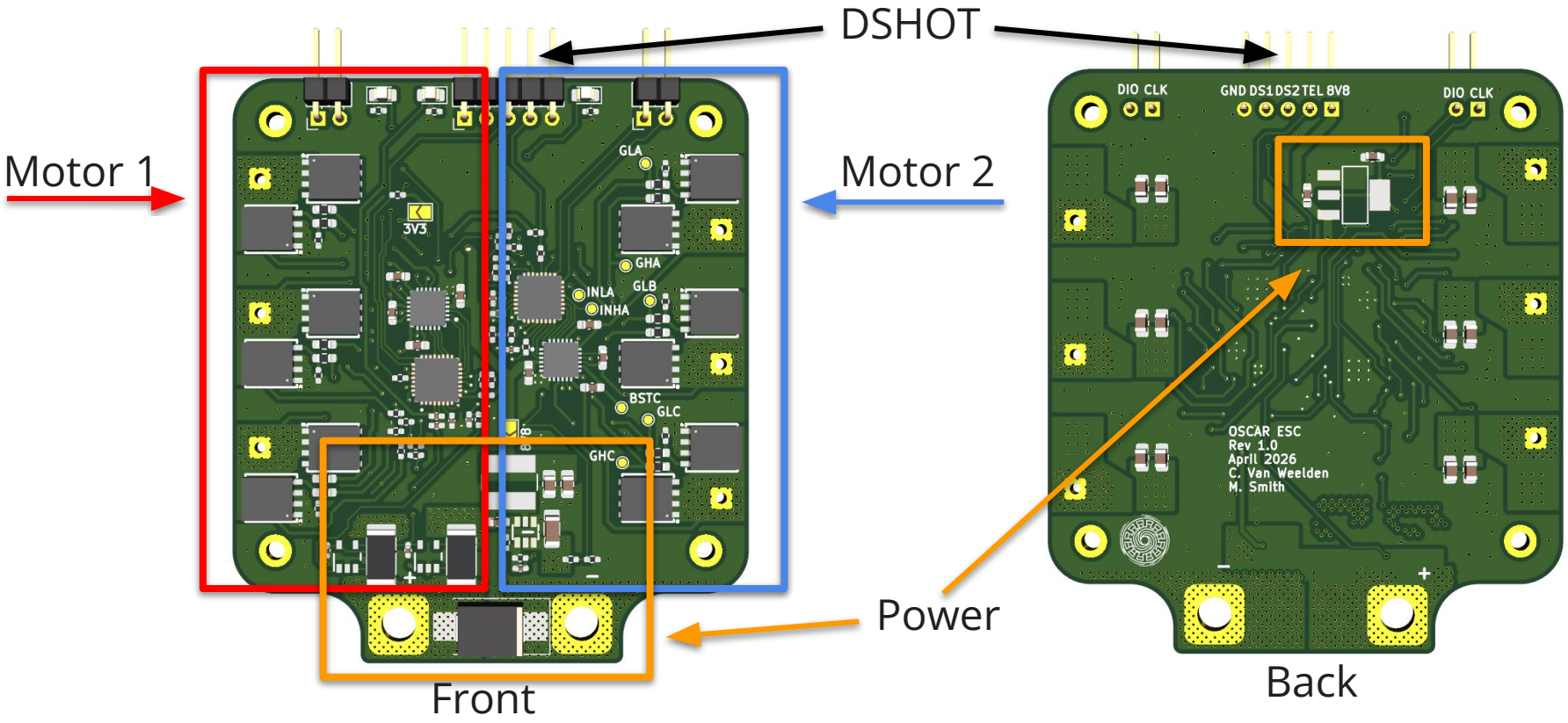
Hardware

- MCU Selection: STM32H753VIT6 → *What is computing everything*?
  - Power Consumption: 3.3V * 110mA = **363mW (Max performance)**
  - Power Consumption: 3.3V * 70mA = **231mW (Standby and measured)**
  - Clock Rate: 480MHz Cortex-M7, Double-precision FPU
  - Memory: 2MB flash, 1MB RAM.
  - I/O: 82 total GPIO ports (6 SPIs, 4 UARTS, 4 I2C Lines, 22 timers available)
- IMU Selection: ICM40609-D → *What is our acceleration and angular velocity data*?
  - Power Consumption: 3.3V * 0.77mA = **2.5mW**
  - Output Data Rate: Up to 32kHz for both accelerometer and gyroscope
  - Handle fast movements gyro supports up to 2000 degrees per second. Accel up to 32gs
  - Accel and Gyro Sensitivity Error: ±0.5%,
- Magnetometer Selection: MMC5983MA→ *What is our orientation*?
  - Power Consumption: 3.3V * 1μA = **3.3μW**
  - Output Data Rate: Up to 1kHz
  - Heading accuracy of ±0.5°

Hardware

- Barometer Selection: ENS220 → How high up in our altitude are we & Temp?
  - Power Consumption: 1.8V * 75μA = **1.35μW**
  - Output Data Rate: Up to 1kHz
  - Relative alttitdue accuracy of ±20cm in air.
- GNSS (GPS) Selection: TESEO-LIV3R→ What is our X and Y position?
  - Power Consumption(Tracking Mode): 3.3V * 21mA = **70mW** Tracking Mode
  - Includes GPS, GLONASS, BeiDou constellations
  - Output Data Rate: Up to 10 Hz
  - position accuracy < 1.8 m CEP (50%).

ESC Front and Back

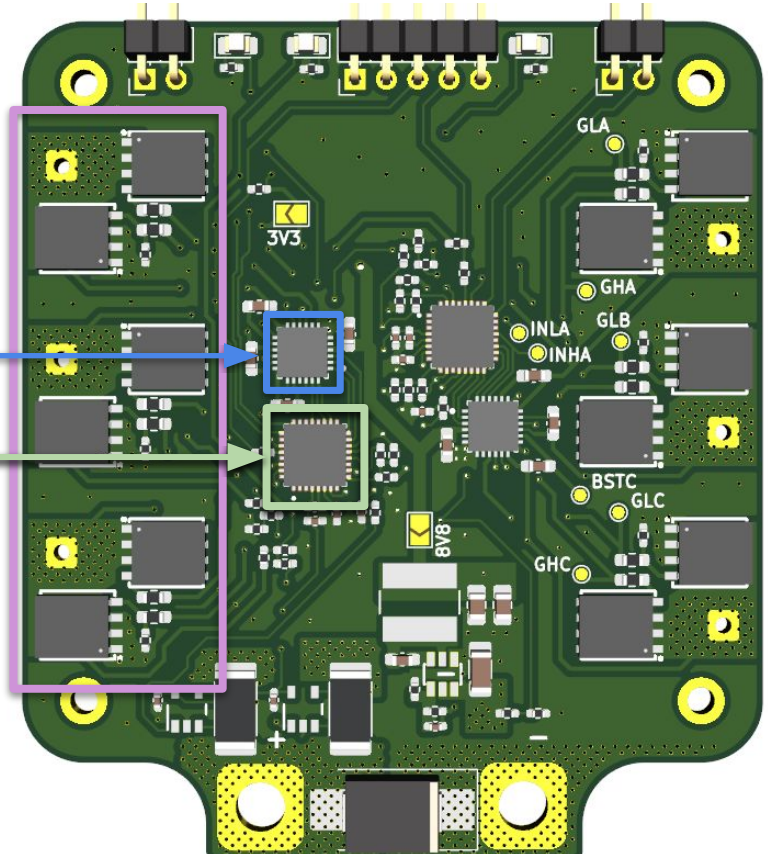


ESC Components

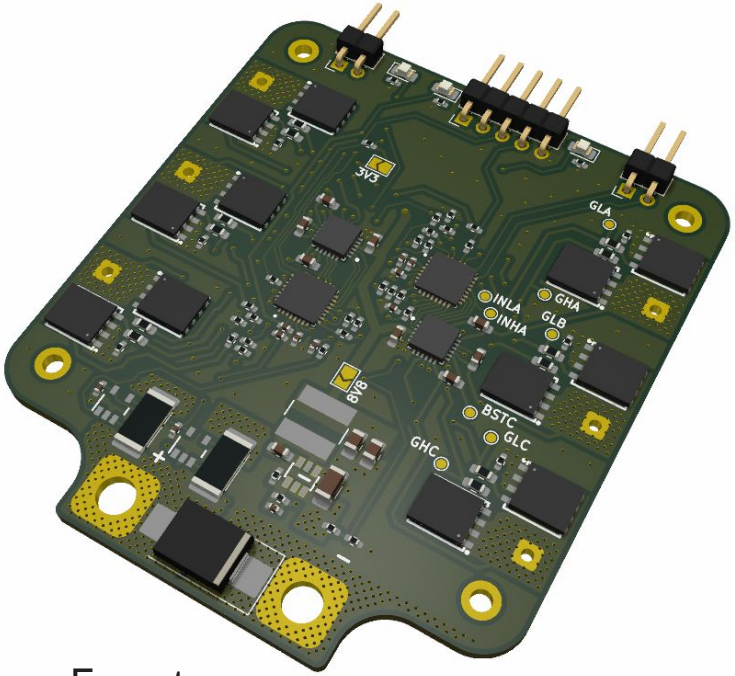
3-Phase Gate driver

Microcontroller

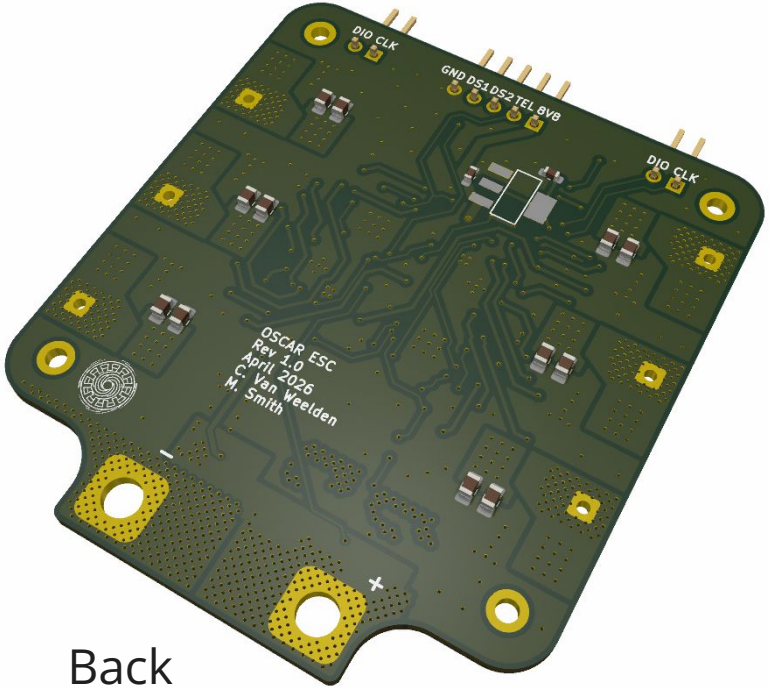
MOSFETs



ESC 3D View

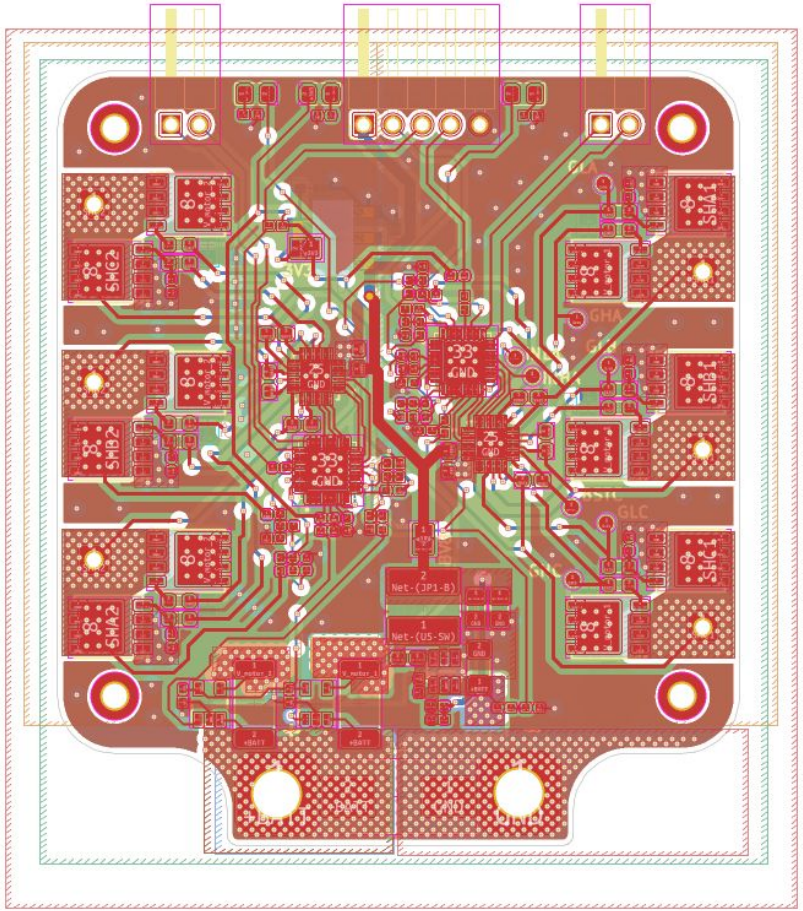


Front

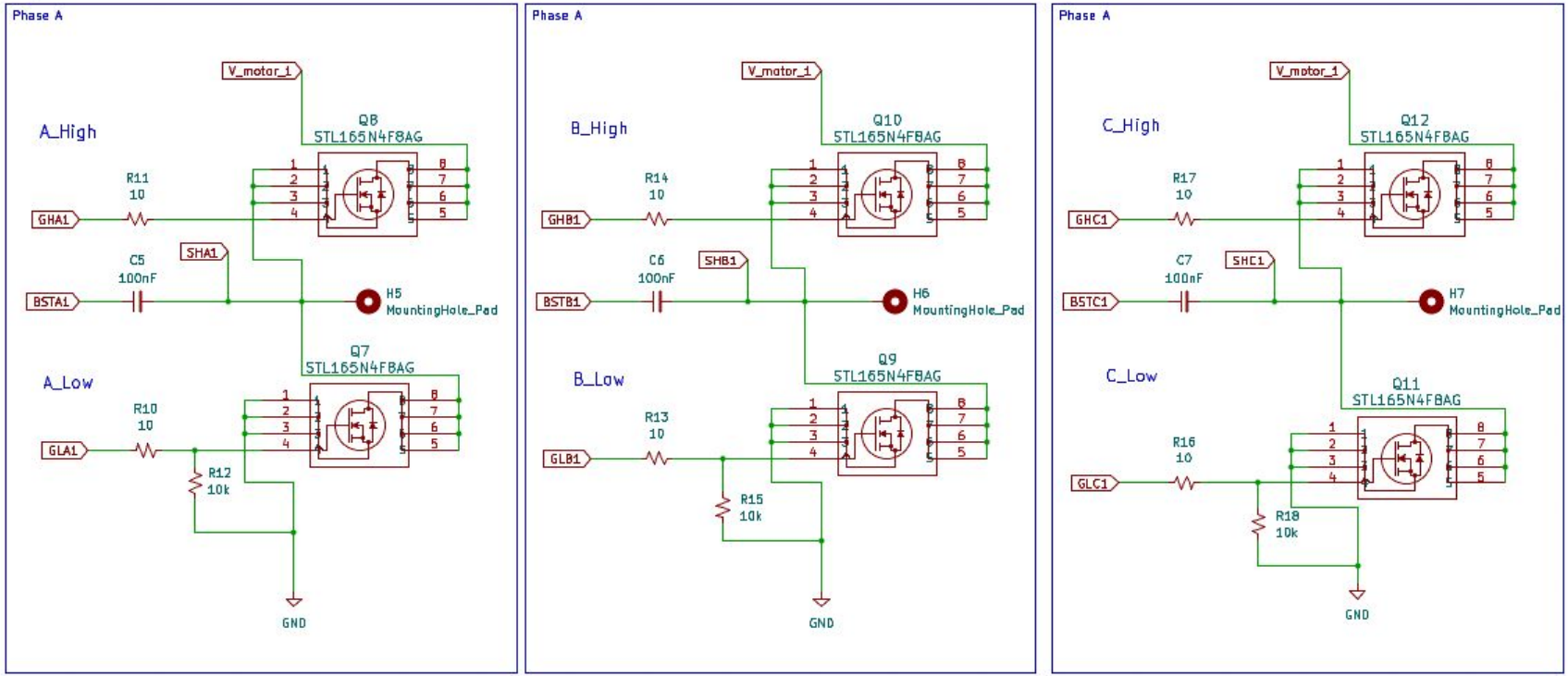


Back

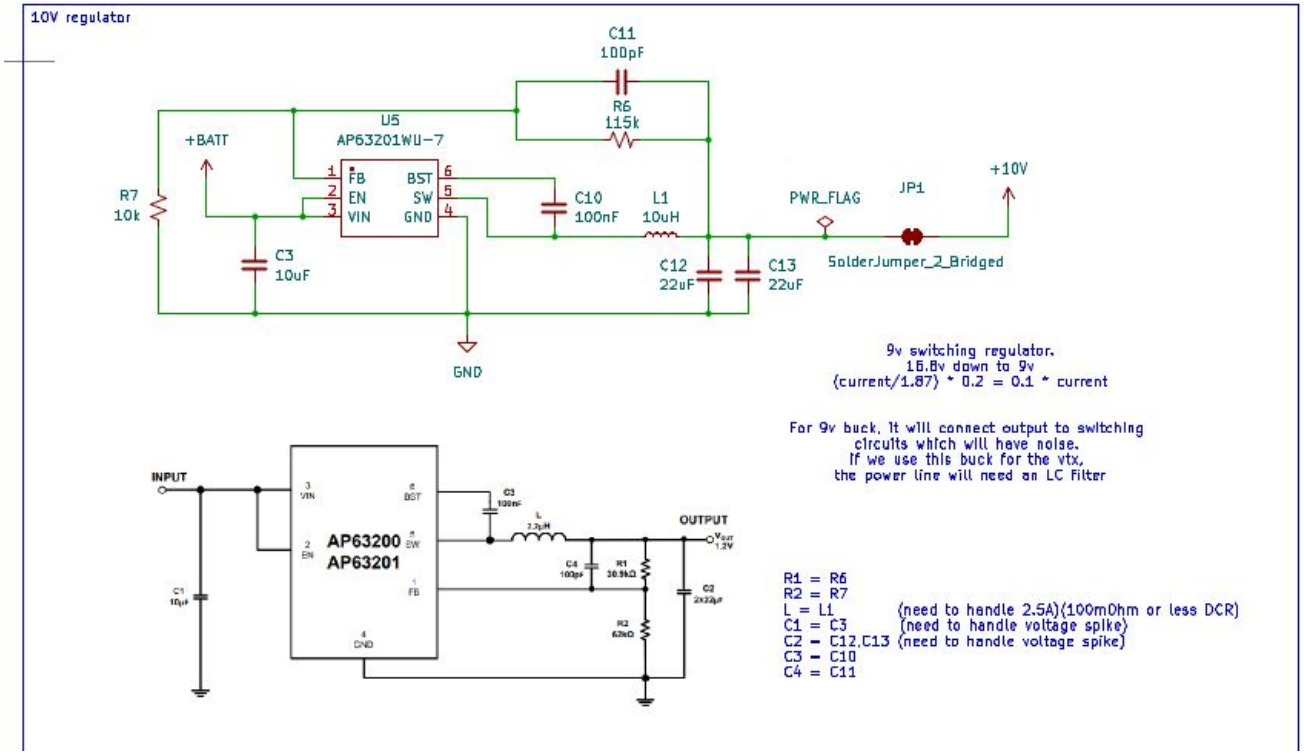
ESC Routing



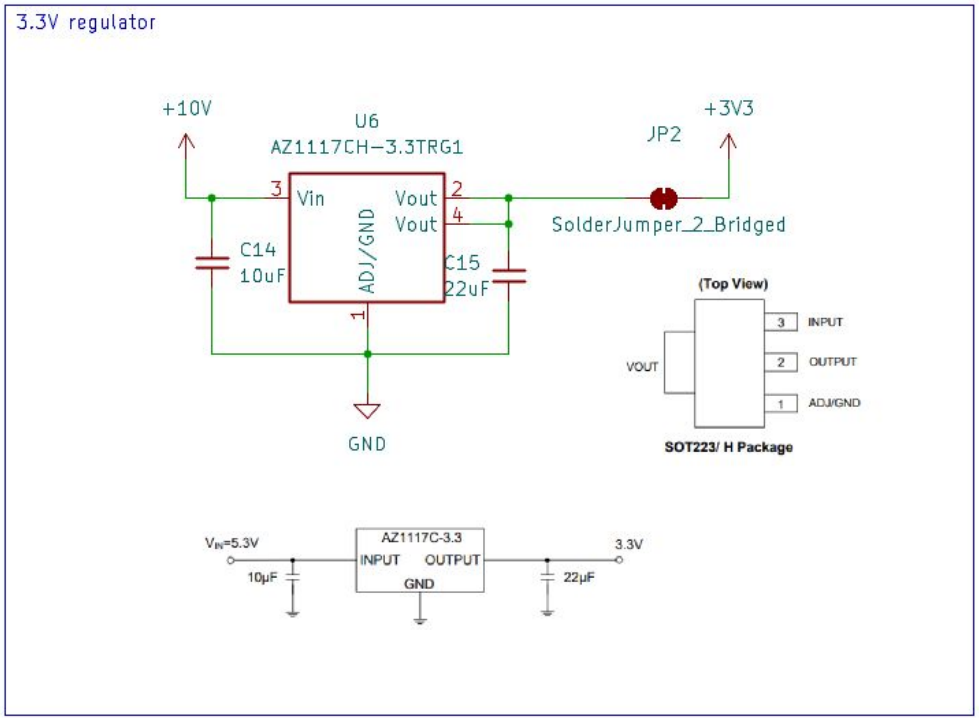
MOSFET Schematic



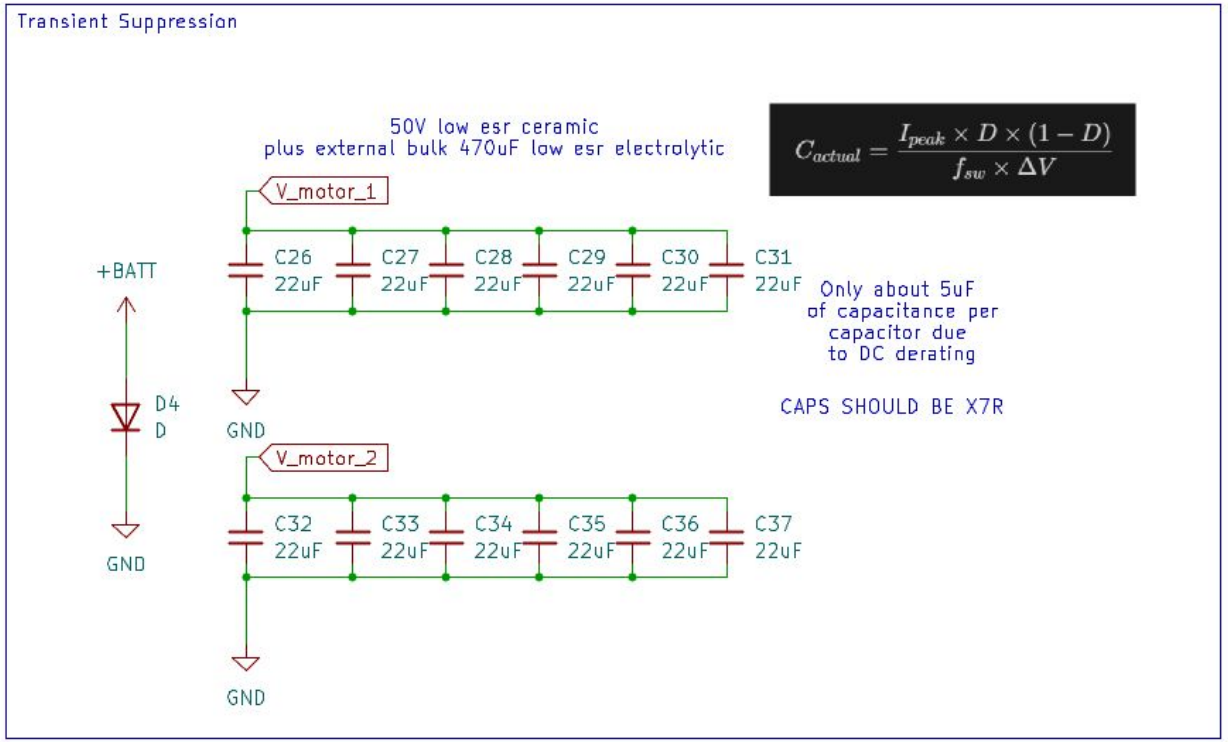
10V Voltage Regulator Schematic



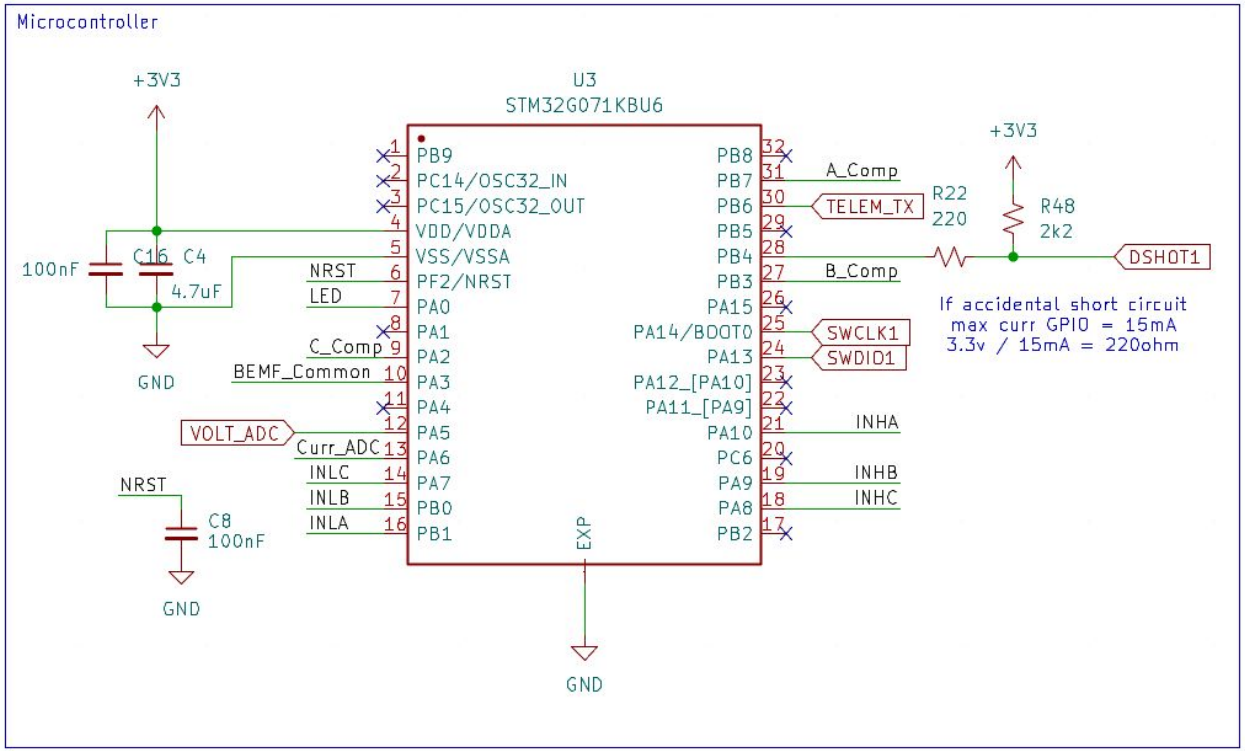
3.3V Voltage Regulator Schematic



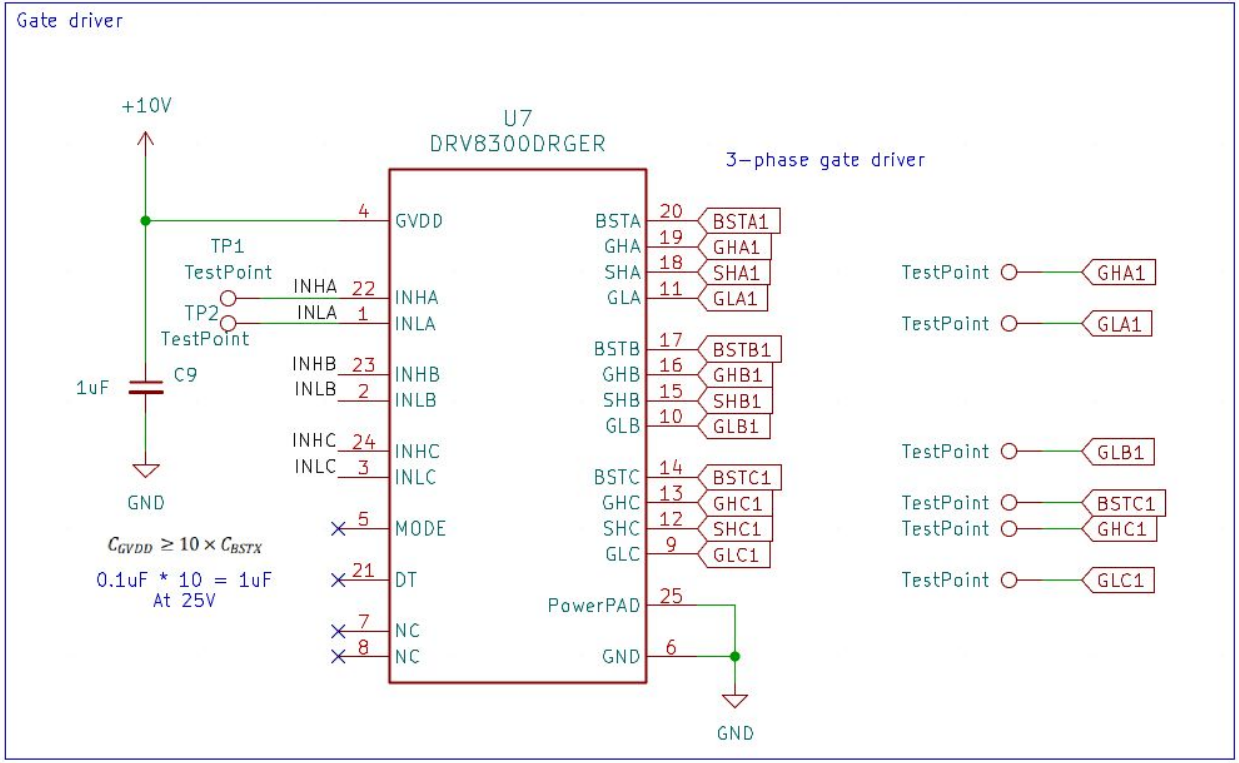
Transient Suppression Schematic



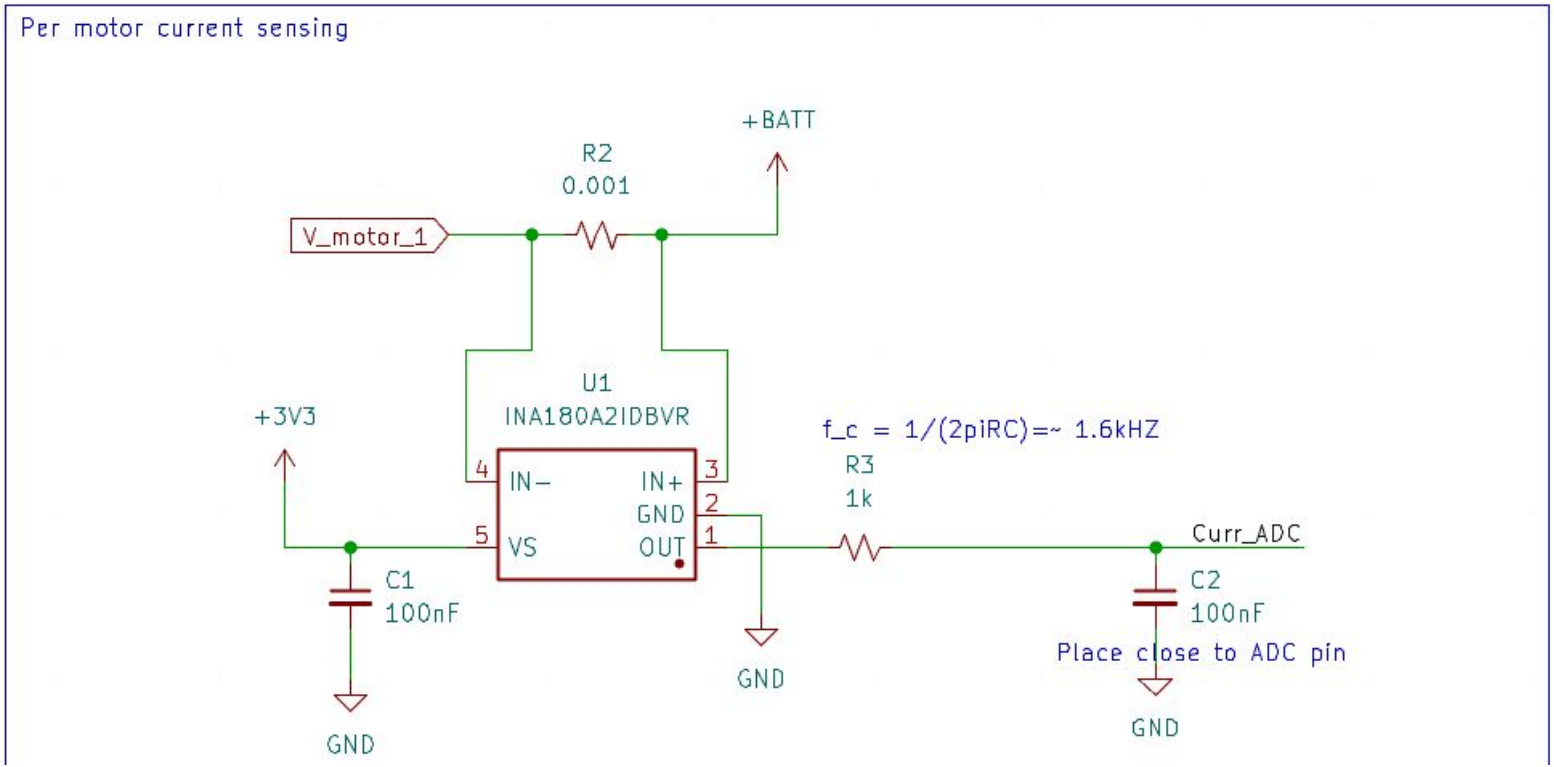
STM32G07 Schematic



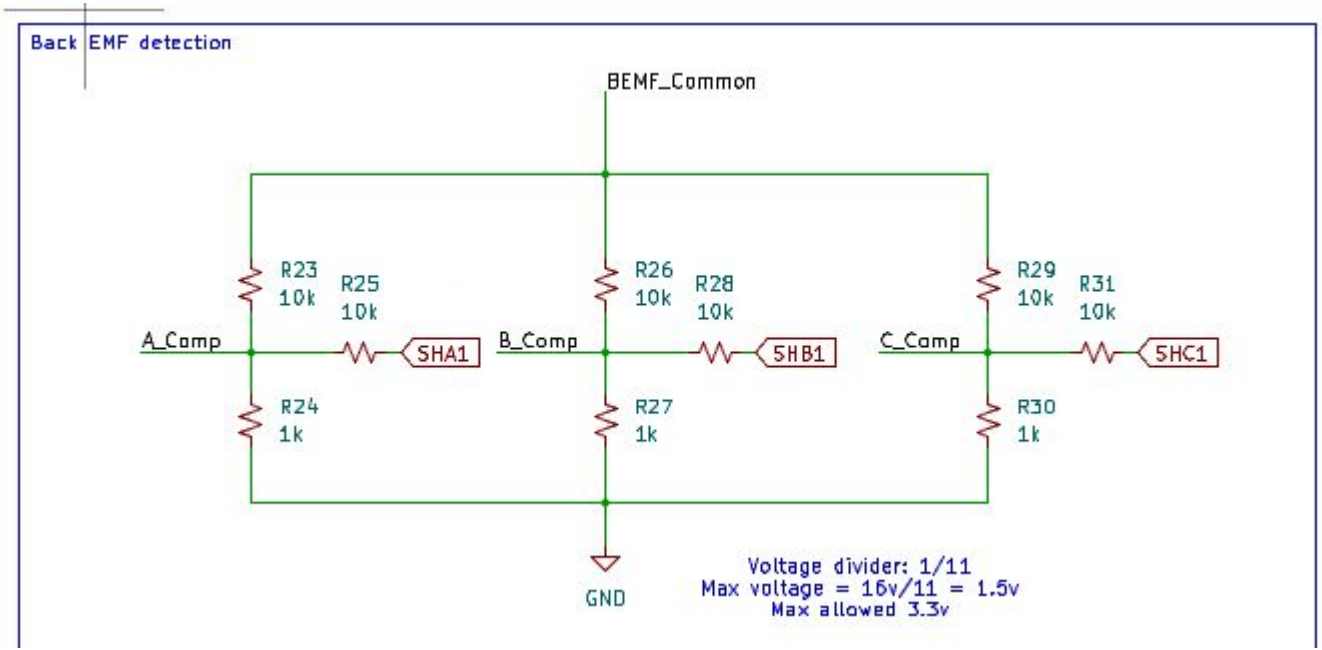
Gate Driver Schematic



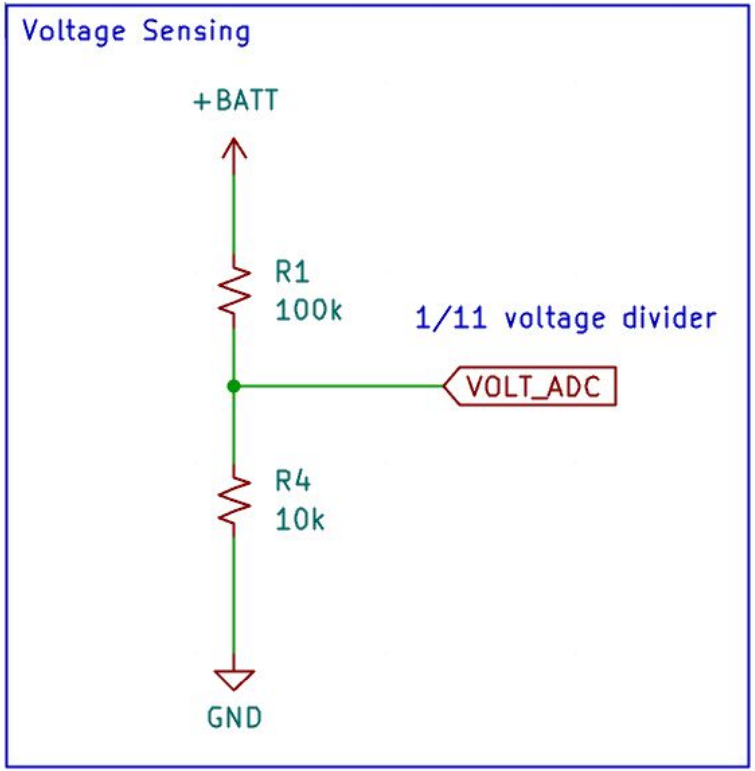
Current Sensing Schematic



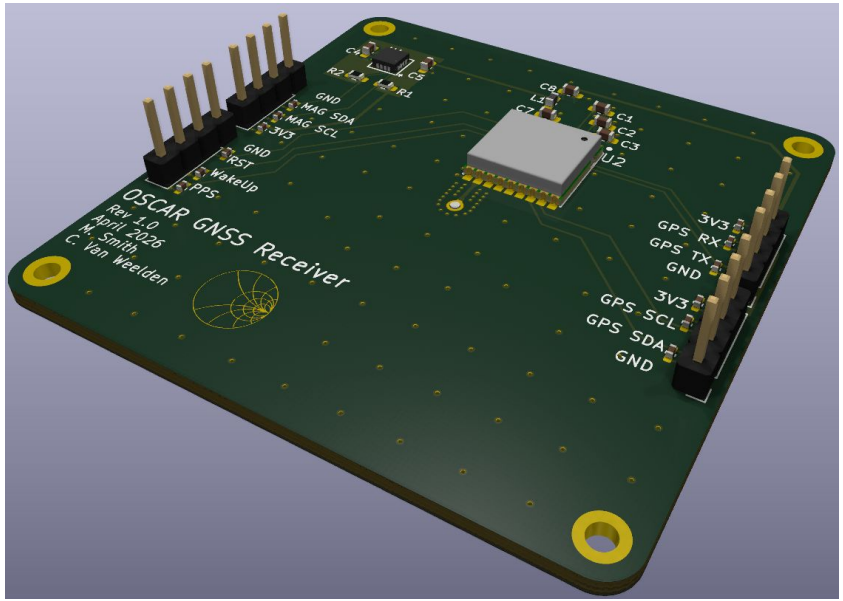
Back EMF Schematic

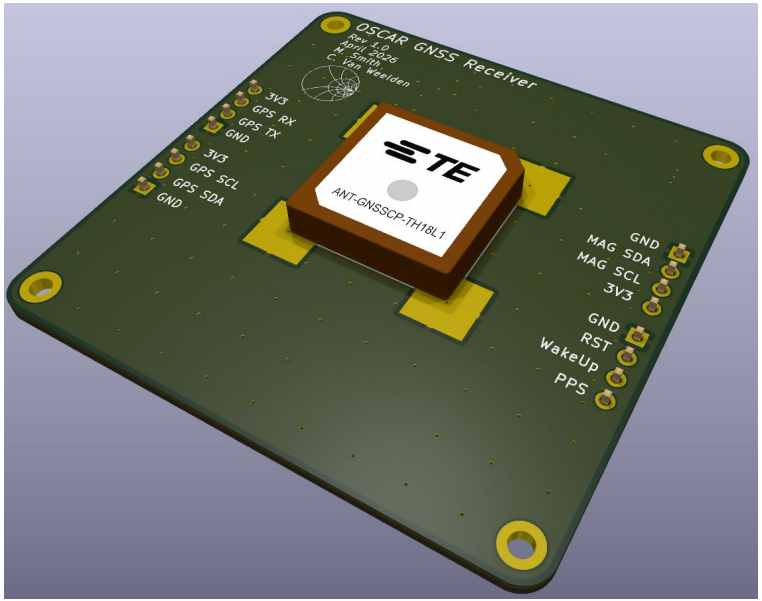


Voltage sensing Schematic

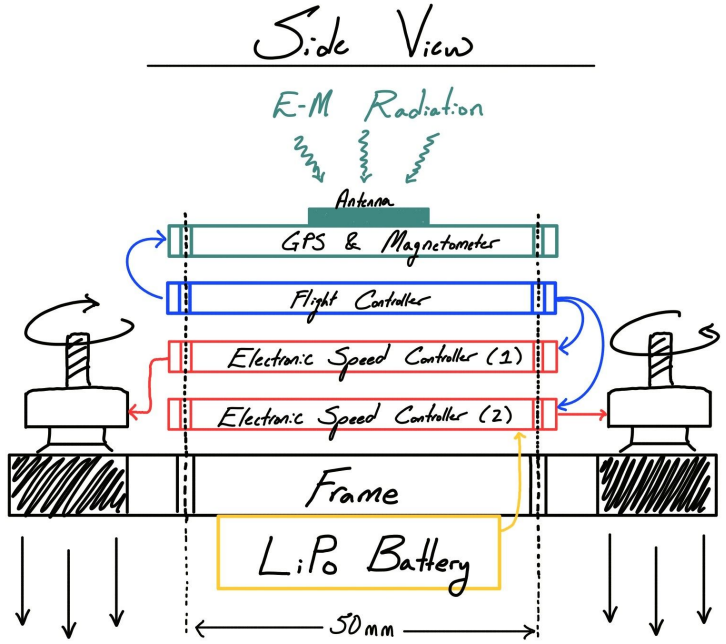


GNSS Layout

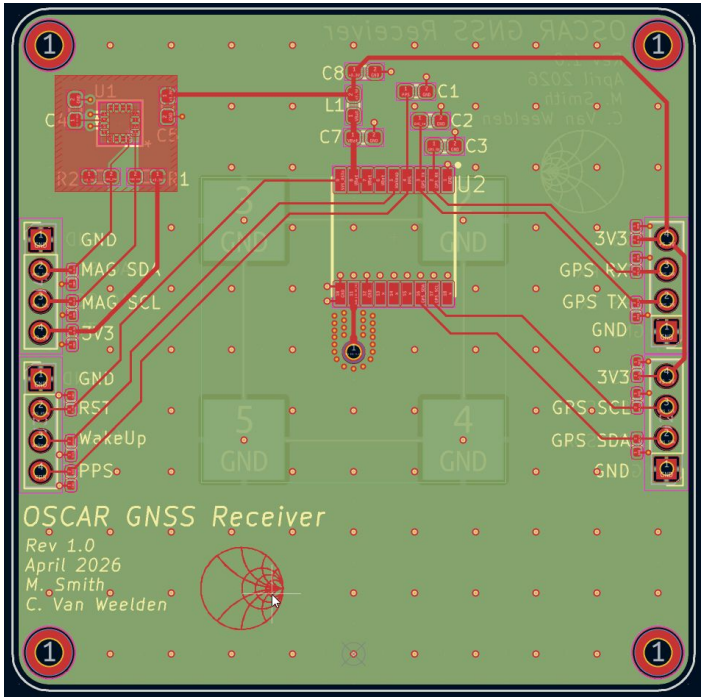

OSCAR GNSS Receiver
Rev 1.0
April 2026
M. Smith
C. Van Weelden
GND
MAG SDA
MAG SCL
3V3
GND
RST
WakeUp
PPS
3V3
GPS RX
GPS TX
GND
3V3
GPS SCL
GPS SDA
GND
U2

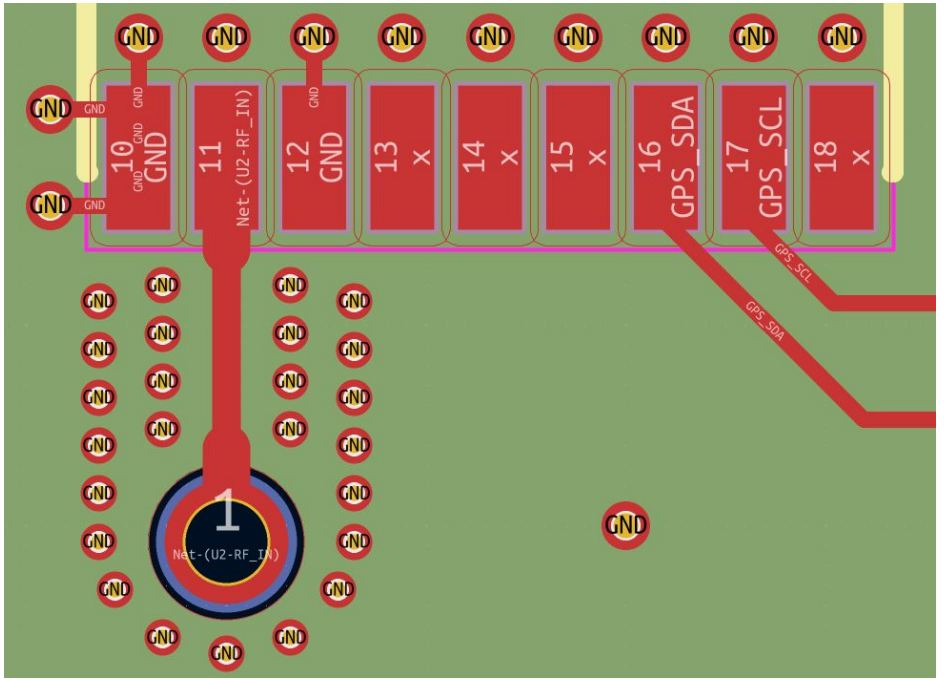

OSCAR GNSS Receiver
Rev 1.0
April 2026
M. Smith
C. Van Weelden
3V3
GPS RX
GPS TX
GND
3V3
GPS SCL
GPS SDA
GND
ANT-GNSSCP-TH18L1
GND
MAG SDA
MAG SCL
3V3
GND
RST
WakeUp
PPS

Side View Sketch

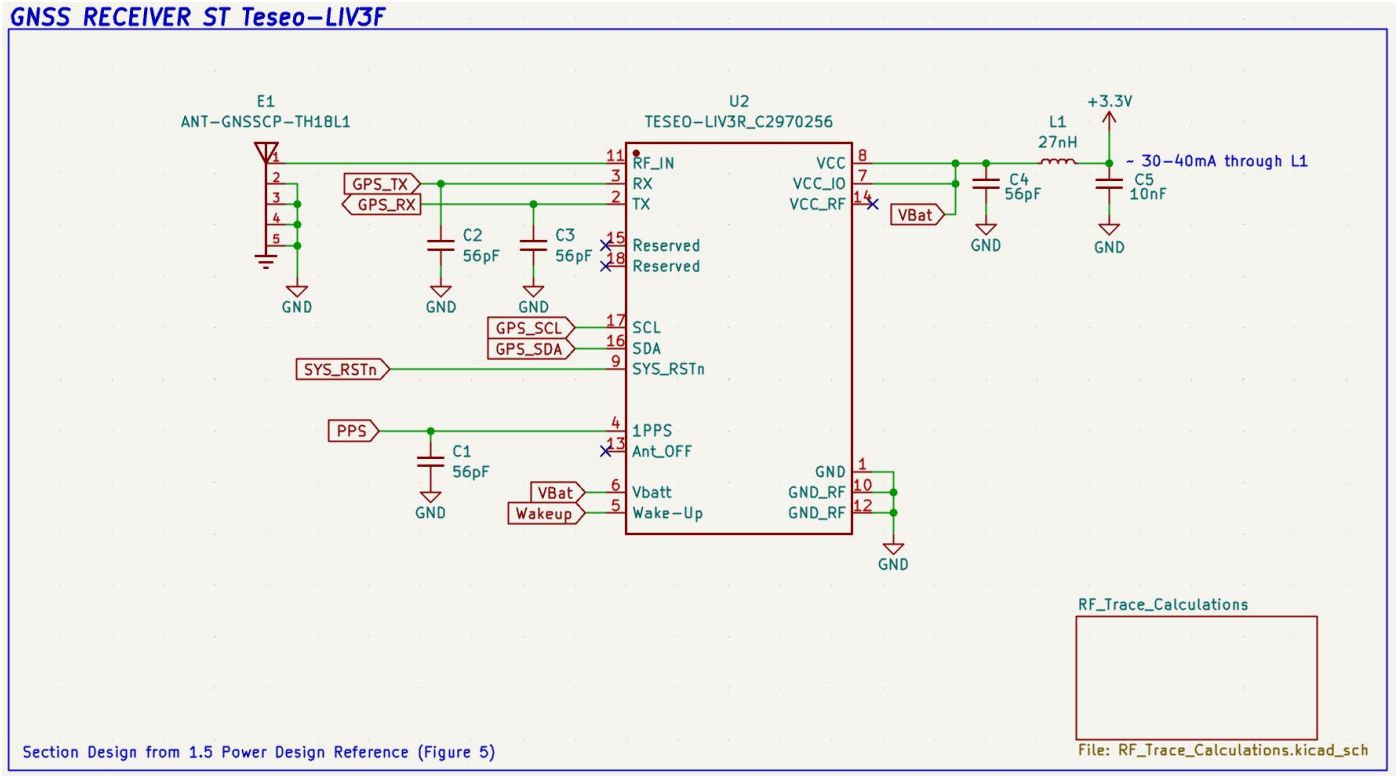


GNSS Layout


OSCAR GNSS Receiver
Rev 1.0
April 2026
M. Smith
C. Van Weelden
GND
MAG_SDA
MAG_SCL
3V3
GND
RST
WakeUp
PPS
3V3
GPS_RX
GPS_TX
GND
3V3
GPS_SCL
GPS_SDA
GND
U2
C1
C2
C3
C7
C8
L1


GND
10 GND
11 Net-(U2-RF_IN)
12 GND
13 X
14 X
15 X
16 GPS_SDA
17 GPS_SCL
18 X
GPS_SCL
GPS_SDA
1
Net-(U2-RF_IN)

GNSS Schematic



GNSS Calculations

50Ω Trace Width Calculation

4-Layer Impedance Control Stackup

Thickness: 0.8mm | 1.0mm | 1.2mm | 1.6mm | 2.0mm

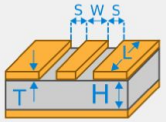
Outer Copper Weight: 1oz | 2oz

Inner Copper Weight: 0.5oz | 1oz | 2oz

1) No requirement Stackup

Layer	Material Type	Thickness	
Top Layer	Copper	0.035mm	
Prepreg	7628*1	0.21040mm	
Inner Layer L2	Copper	0.0152mm	1.1mm H/HOZ with copper
Core>	Core	1.065mm	
Inner Layer L3	Copper	0.0152mm	
Prepreg	7628*1	0.21040mm	
Bottom Layer	Copper	0.035mm	

Transmission Line Type
- Microstrip Line
- Coplanar wave guide
- Coplanar wave guide w/ ground plane
- Rectangular Waveguide
- Coaxial Line
- Coupled Microstrip Line
- Stripline
- Twisted Pair



Substrate Parameters
- εr: 4.4
- tan δ: 0.02
- ρ: 1.72e-08 Ω · m
- H: 0.2104 mm
- T: 0.035 mm
- μ(conductor): 1

Component Parameters
- Frequency: 1.575 GHz

Physical Parameters
- W: 0.35989 mm
- S: 0.2 mm
- L: 3.3 mm

Analyze | Synthesize

Electrical Parameters
- Z0: 50 Ω
- Ang_l: 10.3166 deg

Results
- Effective εr: 2.73228
- Unit propagation delay: 55.2829 ps/cm
- Conductor losses: 0.00541529 dB
- Dielectric losses: 0.0128321 dB
- Skin depth: 1.6632 µm

Trace width of 0.38mm for a Zo = 50Ω

Critical Length Calculation

$c = 3.00 \cdot 10^8$ $= 300000000$

$f = 1.575 \cdot 10^9$ $= 1.575 \times 10^9$

$e_{ff} = 3.102$ $= 3.102$

$L_S = \frac{c}{f} \cdot \frac{1}{\sqrt{e_{ff}}} \cdot \frac{1}{12}$ $= 0.009012360466$

$L_S \cdot 1000$ $= 9.012360466$

Frequency:	1.575	GHz
Period:	0.634921	ns
Wavelength in vacuum:	19.0344	cm
Wavelength in medium:	9.07431	cm
Speed in medium:	1.4292e+08	m/s
er:	4.4	...
mur:	1	

Our Critical Length is less than 9mm

GNSS Calculations

Via Spacing Calculation

The following formula is used to determine the D value:

$$\frac{\lambda_G}{20} \le D \le \frac{\lambda_G}{10}$$

With λ_G, as guided wavelength, defined by this formula:

$$\lambda_G = \frac{3 \times 10^8}{f \times \sqrt{\varepsilon_{r_eff}}}$$

$$L < \frac{c}{20 \cdot f \cdot \sqrt{D_k}}$$

$c = 3.00 \cdot 10^8$	$= 300000000$
$f = 1.575 \cdot 10^9$	$= 1.575 \times 10^9$
$D_k = 4.4$	$= 4.4$
$L_m = \frac{c}{20 \cdot f \cdot \sqrt{D_k}}$	$= 0.004540298044$
$L_{mm} = L_m \cdot 10^3$	$= 4.540298044$

Via Spacing needs to be Center to Center 4.54mm

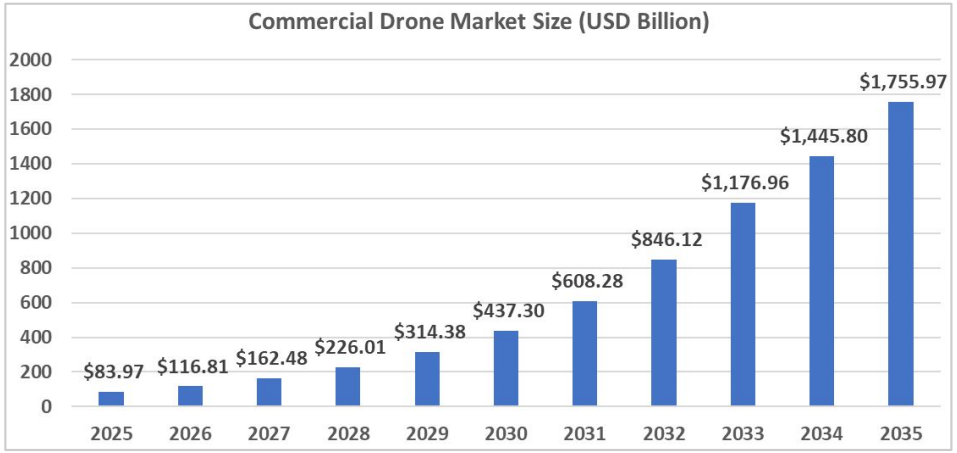
Unused slides

Notes from Class on this

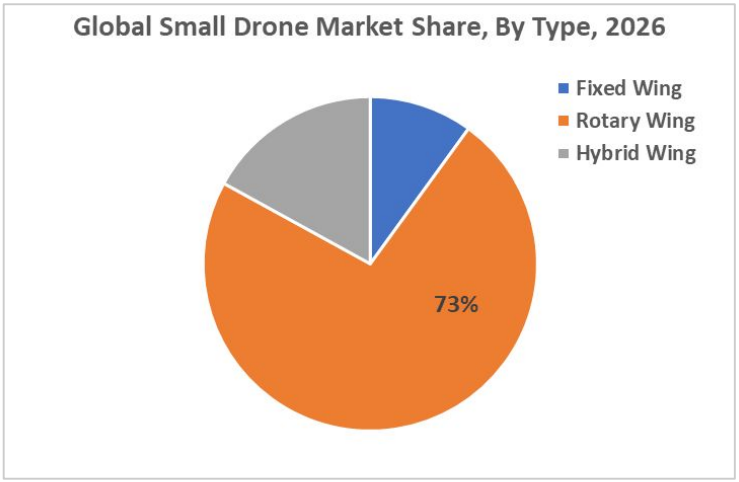
- Focus on storytelling
  - What is the problem you are solving
  - Why is it important
- Have back-up slides that go more in depth
  - Be Clear and Concise - do not need to be too technical overall
- Good Visuals (Duh)
- Thorough design plan
  - METRICS!!!!
- Bring prototypes or embed video into presentation - show your work!!
- It is okay to say “I don’t know” - handle Q&A gracefully
  - Do not let one person dominate the answers
- Practice, Practice, Practice
  - Can Sign up for time slot the week before during class

Background

- Rapidly growing industry
- Overall drone interest
- Increased focus on drone research



https://www.precedenceresearch.com/commercial-drone-market

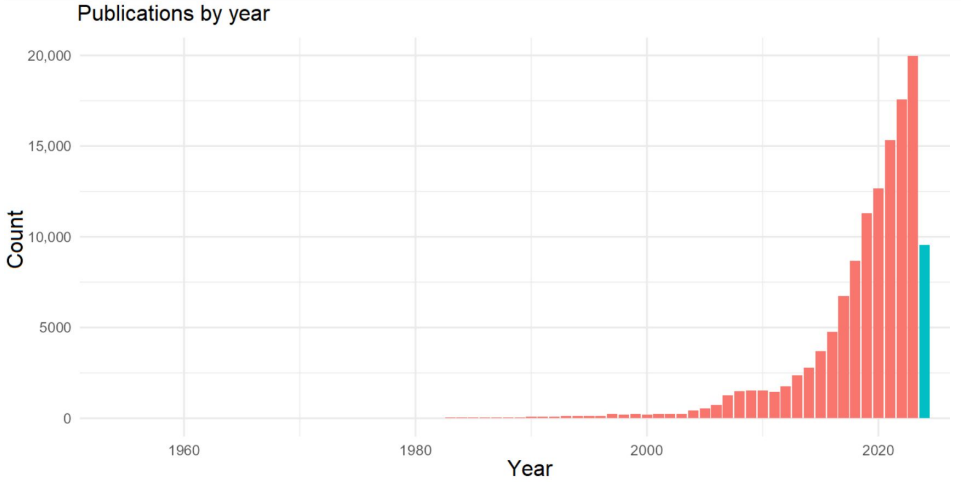


https://www.fortunebusinessinsights.com/small-drones-market-102227

How we fit in

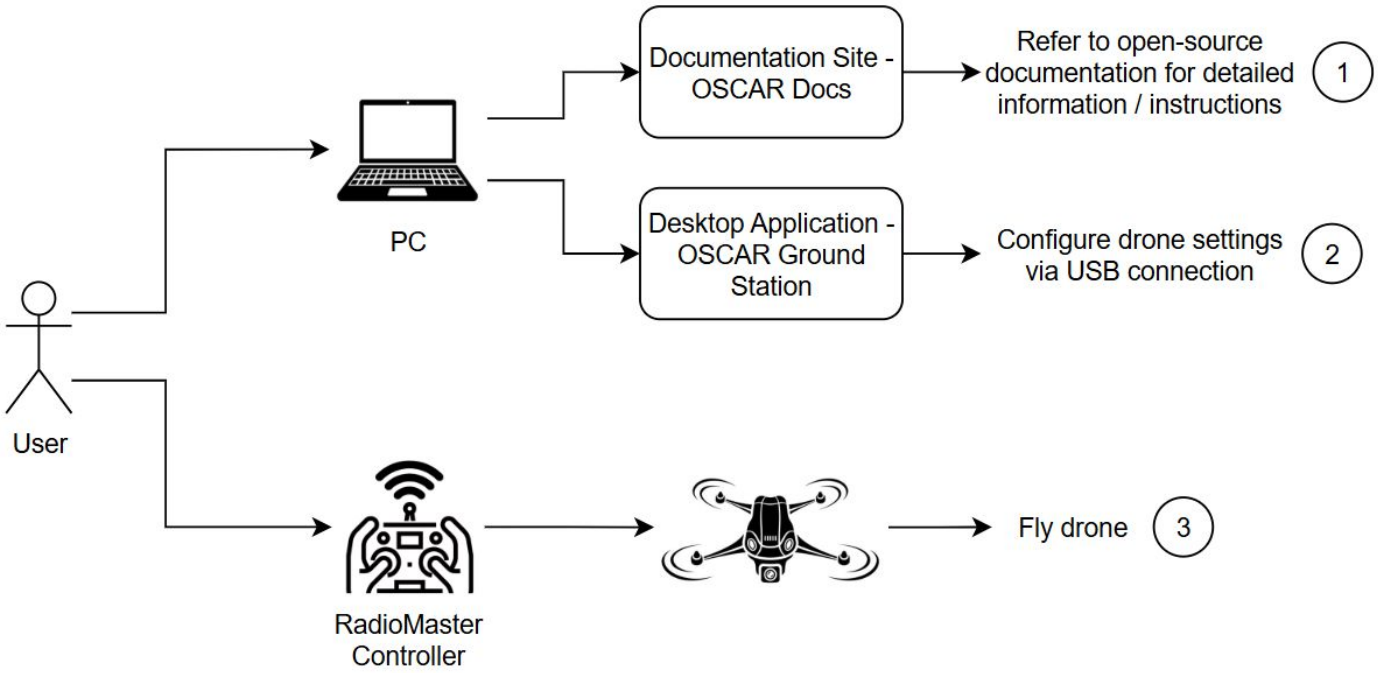
- Rapidly growing market
- Base platform for drone research
- Students looking to learn
- Hobbyists wanting hyper-speific features not commercially available

Bin	Papers (N)	Energy (%)	Communication (%)
1990–1994	478	9.0	15.5
1995–1999	929	11.5	16.3
2000–2004	1332	16.1	39.0
2005–2009	5571	18.5	47.9
2010–2014	9889	17.8	57.6
2015–2019	35,121	21.2	59.5
2020–2024	75,111	24.7	64.1



[1] T. Guzsvinecz and J. Szűcs, “A bibliometric analysis of the trends in UAV research using the Bibliometrix R-tool,” Applied Sciences, vol. 15, no. 21, p. 11305, 2025, doi: 10.3390/app152111305.

Project Use Cases



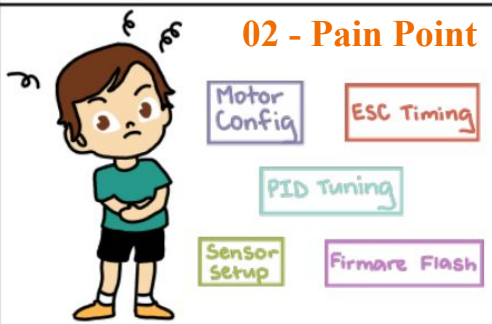
StoryBoard

01 - The Problem



Drone enthusiasts have separate FC and ESC tools with no open source hardware to learn from

02 - Pain Point



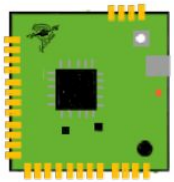
A variety of setup and tuning parameters are all across separate apps with no unified workflow.

03 - The Solution



OSCAR unifies FC and ESC Firmware configuration into a single platform with a desktop app.

04 - Our Design



Our FC includes a IMU, and barometer. Users can optionally attach GPS, magnetometer, and camera, all managed through OSCAR

05 - Value Prop

OSCAR extends beyond Betaflight by combining FC and ESC config, onboard sensor data, and a desktop app.

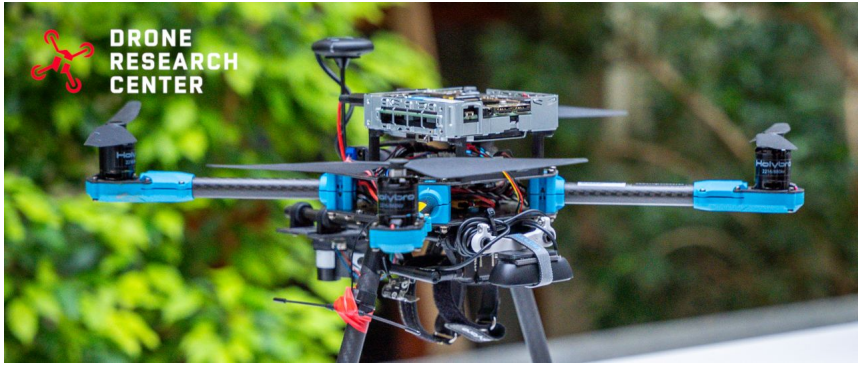
06 - The Outcome



Builders can configure the FC and ESC from the desktop App, and takes flight with a controller.

Users

- Base platform for drone research
- Students looking to learn
- Hobbyists wanting hyper-specific features not commercially available





https://www.sphengineering.com/industries/academic-research